THE BEE BOOK

A comprehensive guide to the Bee plugin for Claude Code

Covering version 4.0.3

50 commands · 39 agents · 41 skills · 10 stacks

by Beecoded

The Bee Book

A comprehensive guide to the Bee plugin for Claude Code.

Version 4.0.3 · Built 2026

Bee is a Claude Code plugin published by Beecoded under the MIT License. This book is a companion reference explaining Bee's philosophy, commands, agents, skills, workflows, and the situations where each piece is useful.

No part of this book assumes you are a Claude Code expert. It starts from first principles, then walks through each capability in depth, then shows you how the pieces combine into real-world workflows.

Table of Contents

The Bee Book . . . 2

The Bee Way 10

What is Bee? . . . 11

The five-word definition . . . 11

What Bee is useful for . . . 11

What Bee is not useful for . . . 11

The short history . . . 12

How to read this book . . . 12

Philosophy: The Five Pillars . . . 13

Pillar 1 — Spec before code . . . 13

Pillar 2 — Tests first, always . . . 13

Pillar 3 — Disk is truth . . . 13

Pillar 4 — Review gates . . . 14

Pillar 5 — Humans commit . . . 14

The Lifecycle: From Idea to Commit . . . 15

Stage 0 — Initialization . . . 15

Stage 1 — Discovery and specification . . . 15

Stage 2 — Planning . . . 15

Stage 3 — Execution . . . 15

Stage 4 — Review . . . 16

Stage 5 — Testing . . . 16

Stage 6 — Commit . . . 16

The whole thing in one page . . . 16

Architecture: Commands, Agents, Skills, Hooks . . . 17

Commands are conductors . . . 17

Agents are specialists . . . 17

Skills are knowledge bases . . . 17

Hooks are the immune system . . . 18

How the four layers interact . . . 18

Getting Started 19

Installation and the First Run . . . 20

Prerequisites . . . 20

Installing the plugin . . . 20

Initializing a project . . . 20
Checking installation . . . 20

Your First Spec . . . 21
Start the discovery . . . 21
The spec emerges . . . 21
Read the spec . . . 21

Your First Phase . . . 22
Plan the phase . . . 22
Execute the phase . . . 22

Your First Review and Commit . . . 23
Review . . . 23
Test . . . 23
Commit . . . 23

The .bee/ Directory Explained . . . 24
Top-level files . . . 24
Spec directories . . . 24
Completed specs . . . 24
Quick tasks and seeds . . . 24
How to read STATE.md . . . 25

Core Concepts **26**

Spec-Driven Development . . . 27
The short definition . . . 27
What Bee's specs look like . . . 27
Phase breakdown . . . 27
The cost . . . 28

TDD as Iron Law . . . 29
Red-Green-Refactor . . . 29
Why strict TDD . . . 29
When TDD doesn't fit . . . 29

Wave-Based Parallel Execution . . . 30
What a wave is . . . 30
Why waves matter . . . 30
The scoped testing trick . . . 30
What to plan for . . . 30

Disk is Truth: State Management . . . 31

What lives on disk . . . 31
What does not live on disk . . . 31
Why this matters practically . . . 31
The rule of thumb . . . 31

Multi-Agent Orchestration . . . 32
The single-agent problem . . . 32
The specialist solution . . . 32
The validator solution . . . 32
The consolidator solution . . . 32

Implementation Modes and Model Tiering . . . 33
The three modes . . . 33
How to choose . . . 33

Complete Command Reference **34**

Setup and Navigation . . . 35

Specification Commands . . . 41

Execution Commands . . . 46

Quality Commands . . . 50

Testing Commands . . . 53

Audit Commands . . . 55

Debug and Recovery Commands . . . 57

Session, Ideas, and Backlog Commands . . . 59

Visualization: The Bee Hive Dashboard . . . 64

Extensibility Commands . . . 65

The Agent Roster **67**

How to Read This Part . . . 68
The general agent contract . . . 68

Implementation Agents . . . 69

Planning and Research Agents . . . 71

Review Agents . . . 74

Audit Agents . . . 78

Debug, Specialized, and Stack-Specific Agents . . . 80

Stack-specific variants . . . 84

Skills, Stacks, and Libraries 85

Core and Standards . . . 86

The core skill . . . 86

Standards . . . 86

Stack Skills . . . 87

Library Skills . . . 88

How Skills Load . . . 89

Workflows and Situations 90

The Full Feature Workflow . . . 91

When to use it . . . 91

The steps . . . 91

Why this order . . . 91

Common variations . . . 91

The Autonomous Ship Workflow . . . 93

When to use it . . . 93

The steps . . . 93

What to watch for . . . 93

The Quick-Task Workflow . . . 94

Default mode — TDD with agents . . . 94

Fast mode — direct edits . . . 94

Review mode — execute + 4-agent review . . . 94

Amending a quick task . . . 94

The Audit and Takeover Workflow . . . 95

The steps . . . 95

Tips for takeovers . . . 95

The Debug and Recovery Workflow . . . 96

Step 1 — Forensics, if something went wrong in a workflow . . . 96

Step 2 — Debug session . . . 96

Step 3 — Health baseline . . . 96

Step 4 — Fix via the normal pipeline . . . 96

The Inherited Project Situation . . . 97
First day . . . 97
First week . . . 97
First month . . . 97
The Multi-Stack Monorepo . . . 98
Init detection . . . 98
Spec scope in monorepos . . . 98
Tips . . . 98
The Crash and Resume Story . . . 99
What gets saved, when . . . 99
How to recover . . . 99
When things really break . . . 99
Situational Recipes . . . 100
I want to add one small thing to an existing phase . . . 100
I want to capture an idea without stopping current work . . . 100
I fixed a bug manually and want a regression test . . . 100
I want to audit only one dimension . . . 100
I want to pause and come back in a week . . . 100
I'm blocked on a decision and want to think with the AI . . . 100
I want to show my teammate what Bee did this week . . . 101
A review agent keeps flagging the same false positive . . . 101
I want Bee to follow a personal convention everywhere . . . 101

Advanced Topics **102**

Configuration Deep Dive . . . 103
The full schema . . . 103
The important knobs . . . 104
Hooks and Validators . . . 105
The 8 lifecycle hooks . . . 105
The 27 validators . . . 105
Customizing Bee . . . 106
Level 1 — config.json . . . 106
Level 2 — .bee/user.md . . . 106
Level 3 — Project-local agents . . . 106
Level 4 — Project-local skills . . . 106
The Bee Hive Dashboard . . . 107

Layout . . . 107
Useful views . . . 107
Endpoints . . . 107

Troubleshooting . . . 108
'execute-phase' fails with 'phase not planned' . . . 108
'execute-phase' keeps failing the same task . . . 108
Review flagging things that are not actually bugs . . . 108
Spec discovery feels like it's going in circles . . . 108
/bee:ship made a decision I disagree with . . . 108
The dashboard won't start . . . 108
I accidentally ran /bee:init on an existing project . . . 108

Anti-patterns . . . 109
Skipping the spec because 'it's obvious' . . . 109
Ignoring review findings because 'they're mostly noise' . . . 109
Committing without /bee:commit . . . 109
Hand-editing TASKS.md without re-reviewing . . . 109
Using /bee:quick --fast for anything meaningful . . . 109
Running /bee:ship on an unreviewed plan . . . 109
Mixing Bee work with untracked hand-written work . . . 110

Appendices **111**

Appendix A: Alphabetical Command Index . . . 112

Appendix B: Agent Index . . . 114

Appendix C: The .bee/ File Reference . . . 116

Appendix D: Glossary . . . 117

Appendix E: Keyboard and Endpoint Reference . . . 119
Bee Hive keyboard shortcuts . . . 119
Bee Hive endpoints . . . 119
Lifecycle hook list . . . 119

Closing Notes . . . 120

Part I

The Bee Way

Philosophy, architecture, and the lifecycle of a feature

Chapter 1

What is Bee?

Bee is a plugin for Claude Code, Anthropic's command-line tool for agentic coding. Where Claude Code gives you an AI pair programmer you can type to, Bee gives that pair programmer a **discipline**: a repeatable, spec-driven workflow in which every feature passes through the same cycle of specification, planning, test-first implementation, multi-agent review, testing, and deliberate commit. Bee's job is to turn loose conversation with an AI into shipped software that you would be happy to commit to your main branch.

Bee is not a framework for building AI systems. It is not a code-generation library. It is not a replacement for your test runner, your linter, or your CI. It is a **workflow conductor** that orchestrates specialized AI agents against your existing toolchain, keeps project state on disk so it cannot get lost between sessions, and enforces the kind of process hygiene that senior engineers apply instinctively but that is easy to forget in a 2am debugging session.

The five-word definition

> *Bee: spec · plan · execute · review · commit.*

This is the lifecycle that runs under every Bee-led feature. Each stage has its own commands, its own agents, and its own artifacts on disk. You can drive the pipeline one step at a time, batch several phases together, or hand the whole thing to Bee in fully autonomous mode and walk away.

What Bee is useful for

- Greenfield features that need a clean specification, phase breakdown, and disciplined TDD implementation.
- Brownfield work where you want to add one feature to a large existing codebase without drifting from local conventions.
- Inheriting a 'vibe-coded' project — running */bee:audit* to get a structured view of everything that is wrong, then converting findings into actionable specs with */bee:audit-to-spec*.
- Debugging long-running bugs that span multiple files or layers, where a structured hypothesis-driven investigation with */bee:debug* is more effective than poking around.
- Teams that want an AI assistant whose output is predictable, reviewable, and leaves a paper trail on disk that any teammate can read.

What Bee is not useful for

- Pure Q&A; sessions where you just want to ask Claude a single question — use vanilla Claude Code.
- One-line edits where the cost of running a pipeline exceeds the cost of the edit itself. For tiny changes, Bee offers */bee:quick --fast*, but for truly trivial edits you probably don't need any assistance.
- Projects where you are not willing to keep test infrastructure in place. Bee enforces TDD; if you refuse to write or run tests, Bee's main value proposition collapses.

The short history

Bee started as a handful of Claude Code slash commands for capturing a spec and running a phase. Through versions 1, 2, and 3 it grew a spec-review loop, multi-agent review, audit agents, and autonomous pipelines. Version 4 — the version this book covers — introduced debug intelligence, health baselining, adaptive error recovery, a live dashboard, and rounded out the command set at 50 commands, 39 agents, and 41 skills. Version 4 is also the first where Bee is genuinely usable without memorizing the command list: */bee:do*, */bee:next*, and */bee:progress* make the workflow self-guiding.

How to read this book

Parts I through III explain the philosophy, the architecture, and the core concepts you need to operate Bee confidently. If you are new to Bee, read them in order. Part IV is the complete command reference — one chapter per command group, one entry per command, treated as a lookup. Part V is the same treatment for agents. Part VI covers skills, stacks, and libraries. Part VII is the most practical section: ready-to-run workflows and situational recipes for the kinds of problems you actually face. Part VIII covers advanced topics, troubleshooting, and anti-patterns. The appendices are flat indexes you can jump into without context.

Chapter 2

Philosophy: The Five Pillars

Bee's behaviour is not an accident. Every command, every agent, every validation hook exists to uphold a small set of principles. You can learn Bee by memorizing its commands, but you will use Bee *well* only once you have internalized these principles. They explain why Bee refuses to do certain things and why it insists on others.

Pillar 1 — Spec before code

Every feature begins with a specification written in human language before any code is written. The spec describes *what* the feature should do in terms of behaviour, inputs, outputs, success criteria, and constraints. It never describes *how*. Code-free specifications force you to reason about the problem, not the implementation, and they make it possible for a different Bee session (or a human reviewer) to evaluate implementation choices against a fixed target.

The spec is amended as understanding grows, but every amendment is a deliberate act with its own review gate. Silent scope creep is the single most common way AI-assisted work goes off the rails, and a living, explicit spec is Bee's primary defence against it.

Pillar 2 — Tests first, always

Bee enforces test-driven development as an iron law, not a recommendation. Before an implementer writes a single line of production code, it must write a failing test (the Red phase). Then it writes the minimum code that makes the test pass (Green). Only then may it refactor, and only if all tests stay green. This cycle is not a cultural suggestion — it is enforced by a *SubagentStop* validator that rejects implementer output which cannot demonstrate a red-then-green transition.

TDD in Bee is not about dogma. It is about *evidence*. When Bee says a task is done, you want the guarantee that a test exists, that the test failed without the change, and that the test passes with the change. Anything less is faith, and AI-assisted work runs on fumes when it runs on faith.

Pillar 3 — Disk is truth

Conversation memory is ephemeral. Context windows fill up, sessions crash, and agents forget. Bee treats the project's *.bee/* directory as the single source of truth for everything that matters: the current spec, the phase breakdown, task status, review findings, decisions made, learnings captured. If it's not on disk, it didn't happen.

This has several practical consequences. You can crash mid-phase and recover by running */bee:resume*. You can pause at the end of the day, come back next morning, and pick up where you left off. You can hand a project from one teammate to another without explaining anything verbally — they read *.bee/STATE.md* and they know exactly where the feature is. The cost is that Bee writes a lot of markdown. The benefit is that Bee never loses anything important.

Pillar 4 — Review gates

Every material change goes through at least one review gate before it is considered done. For a full feature this means */bee:plan-review* on the plan, */bee:review* on the code, and */bee:review-implementation* on the whole spec before archiving. Each gate uses multiple specialized agents in parallel, and then a *finding-validator* agent re-checks every finding against the real code to filter out hallucinations.

The multi-agent-then-validate pattern is how Bee fights one of the hardest problems in AI-assisted coding: plausible-sounding feedback that is simply wrong. A single agent review is easy to hallucinate. Four agents reviewing in parallel produce more findings but also more false positives. Adding a dedicated validator that re-reads the code before each finding is acted on gives you the coverage of multiple specialists without the noise.

Pillar 5 — Humans commit

Bee never commits on its own. Not under */bee:ship*, not under */bee:autonomous*, not under any circumstance. The reasoning is simple: a commit is a durable act. It enters your git history, it flows into CI, it may trigger deploys. Bee's job is to prepare a commit that you, the human, are happy to make; your job is to press the button. */bee:commit* surfaces the diff summary and a suggested message, but the final decision is always yours.

> *If Bee ever seems about to auto-commit, stop. That is a bug.*

Chapter 3

The Lifecycle: From Idea to Commit

Bee's lifecycle is the backbone of the entire system. Every command fits somewhere along this chain, and understanding the chain makes the command list feel natural instead of arbitrary. This chapter walks the lifecycle end-to-end once, in plain language, so the rest of the book has a shared mental model to refer back to.

Stage 0 — Initialization

A project becomes a Bee project when you run */bee:init*. Bee looks at your package.json, composer.json, and build configuration, detects which stack you are on (Laravel+Vue, Next.js, NestJS, and so on), and creates a *.bee/* directory in your repo root. That directory holds *config.json* (your preferences), *STATE.md* (the current workflow state), *CONTEXT.md* (extracted patterns from your codebase), and eventually many more files as specs accumulate.

Stage 1 — Discovery and specification

You have an idea: 'we need role-based access control for the admin panel.' You run */bee:new-spec*. Bee launches a *spec-shaper* agent that asks you questions in an adaptive back-and-forth: who are the roles, what are the permissions, how do they inherit, what happens at login, how are roles stored, what do you want in v1 and what can wait. When it has enough information, a *spec-writer* agent produces *spec.md*, *phases.md*, and a requirements trace in *ROADMAP.md*. A *spec-reviewer* agent then checks for gaps and contradictions. If it finds issues, Bee loops back to you with questions.

The output of Stage 1 is a complete, reviewed specification split into phases, each phase describing a concrete deliverable that can be built independently and tested in isolation.

Stage 2 — Planning

You run */bee:plan-phase 1* for the first phase. A *researcher* agent scans your codebase for existing patterns that the implementation should follow — your router conventions, your testing framework, your authorization helpers. A *phase-planner* decomposes the phase into concrete tasks and groups them into *waves*: tasks in wave 1 have no dependencies and can run in parallel; tasks in wave 2 depend on wave 1; and so on. A *plan-reviewer* double-checks the plan against the spec.

The output is *TASKS.md* — a checklist with each task annotated by wave number, research findings, test scenarios, and acceptance criteria.

Stage 3 — Execution

You run */bee:execute-phase 1*. Bee spawns implementers in parallel, one per task in wave 1. Each implementer owns a slice of the phase; they write failing tests, make them pass, refactor, and mark their task done. Because Bee uses scoped testing, each implementer runs only the tests relevant to its own task — this is how four parallel implementers don't stomp on each other. When wave 1 completes, the conductor runs the full test suite once as a sanity check, then starts wave 2.

The output of Stage 3 is a completed phase: code changes in your working directory, a green test suite, task checkmarks in *TASKS.md*, and a STATE.md that records what was decided.

Stage 4 — Review

You run */bee:review*. Four specialist agents scan the phase in parallel: *bug-detector* (logic and security), *pattern-reviewer* (project conventions), *plan-compliance-reviewer* (did we actually implement the plan), and *stack-reviewer* (framework best practices). Their findings go to *finding-validator*, which re-reads the code to filter hallucinations. The survivors are written to *REVIEW.md*, grouped by severity. If the configuration has the review loop enabled, Bee will automatically apply fixes and re-run the review until clean.

Stage 5 — Testing

Automated tests already cover each task because of TDD. */bee:test* adds *manual* test scenarios — the kind you would give to a QA partner: 'log in as an editor, open the admin panel, try to delete a user, verify the button is disabled and the API returns 403.' Bee generates these scenarios from the spec and walks you through them. For UI-heavy phases, */bee:test-e2e* can generate and run Playwright tests with page-object-model structure.

Stage 6 — Commit

You run */bee:commit*. Bee summarizes the diff, proposes a commit message drawn from the spec and phase metadata, and asks you to confirm. You say yes, Bee runs git commit, and the phase is done. Repeat stages 2 through 6 for each subsequent phase. When all phases are committed, you run */bee:complete-spec* or */bee:archive-spec* to close the spec out and reset STATE.md.

The whole thing in one page

```
/bee:init                # once per project
/bee:new-spec            # describe the feature
loop for each phase:
    /bee:plan-phase N    # research + decompose + review
    /bee:execute-phase N # parallel TDD in waves
    /bee:review          # four specialists + validator
    /bee:test            # manual + optional e2e
    /bee:commit          # human-approved commit
/bee:review-implementation  # final cross-phase sanity check
/bee:complete-spec          # archive and reset
```

Chapter 4

Architecture: Commands, Agents, Skills, Hooks

Bee has four kinds of building blocks. They are layered so that each layer has exactly one job. Understanding these layers is how you go from 'Bee user' to 'Bee power user', because it explains why certain things are commands and others are agents, and how to customize Bee for your own project.

Commands are conductors

A command (like */bee:plan-phase* or */bee:review*) is a *conductor*. It does not do the work itself. It reads the current state from disk, decides which agents to spawn and in what order, passes them their inputs, collects their outputs, validates them, and writes the result back to disk. Conductors are written as markdown files in *plugins/bee/commands/*; they are executed by Claude Code's slash command runtime.

The reason this layer exists separately is determinism. A conductor is a script: it runs in a known order, it reads and writes known files, and its decisions are logged. Even though it is written in natural language, its structure is almost like pseudocode.

Agents are specialists

An agent (like *implementer*, *bug-detector*, *security-auditor*) is a focused worker with one job and a tightly-specified output format. Agents are spawned by conductors via the Task tool; each one runs in its own sub-session with its own context window and its own tool access. Agents are written as markdown files in *plugins/bee/agents/* with a YAML frontmatter that specifies which tools they can use and what model tier they should run on.

The agent-per-job model is what lets Bee scale to 39 specialists. Each agent is small enough to understand and improve independently. If you find that *security-auditor* is missing a class of vulnerability, you can edit that one file without touching anything else.

Skills are knowledge bases

A skill is a bundle of domain knowledge that gets injected into an agent's context when relevant. Bee has skills for the *core* rules, for each supported stack (*nextjs*, *laravel-inertia-vue*, and so on), for popular libraries (*prisma*, *tailwind-v4*, *shadcn-ui*), and for quality standards (*testing*, *frontend*, *backend*, *auth*). Skills load conditionally — the *prisma* skill only loads if Prisma is in your *package.json*, so agents aren't weighed down by knowledge irrelevant to the project.

Skills are the mechanism by which Bee knows *how your stack does it*. They are the reason *implementer* produces idiomatic Laravel on a Laravel project and idiomatic Next.js on a Next.js project without any per-project configuration beyond stack detection.

Hooks are the immune system

A hook is a script that runs automatically on Claude Code lifecycle events: *SessionStart*, *PostToolUse*, *PreToolUse*, *SubagentStart*, *SubagentStop*, *Stop*, *PreCompact*, *SessionEnd*. Bee uses hooks for the things that must happen regardless of what you are doing: auto-linting after writes, injecting user preferences into every sub-agent session, validating that agents conform to their output format, warning you if a session ends with unreviewed phases, and so on.

The most important hook is *SubagentStop*. It runs 27 role-specific validators that check agent output against their contract: 'did the implementer show a red-then-green test transition', 'did the bug-detector produce findings with the required fields', 'did the auditor use the correct finding ID prefix'. If a validator rejects an agent's output, the agent is told to re-do the work until the output is valid. This is Bee's last line of defence against agents drifting from their specifications.

How the four layers interact

Layer	Example	What it does	Where it lives
Command	/bee:execute-phase	Conducts the workflow	plugins/bee/commands/
Agent	implementer	Does one unit of work	plugins/bee/agents/
Skill	prisma	Provides domain knowledge	plugins/bee/skills/libraries/
Hook	SubagentStop	Enforces invariants	plugins/bee/hooks/

A typical flow: you run */bee:execute-phase 1*. The command reads *TASKS.md*, identifies wave 1 tasks, and spawns four *implementer* agents in parallel. Each implementer's *SubagentStart* hook fires, injecting your preferences from *.bee/user.md*. The implementers load the *core* skill and their stack skill automatically. As they write code, the *PostToolUse* hook lints each file. When each implementer finishes, the *SubagentStop* hook runs the implementer validator, which checks for the red-then-green transition and the correct task completion format. If valid, the command proceeds to wave 2. If not, the agent re-runs until it produces valid output.

This sounds complicated. In practice, you never think about it. You type */bee:execute-phase 1*, and a few minutes later you have a phase's worth of working, tested code in your working directory.

Part II

Getting Started

Installing Bee and shipping your first feature

Chapter 5

Installation and the First Run

Prerequisites

Bee is a plugin for Claude Code, so you need Claude Code installed and working on your machine. Bee's own runtime requirements are light: Node.js 18+ (the dashboard uses the built-in HTTP server with no external dependencies) and whatever language runtime your project already uses.

On macOS, notifications use AppleScript; on Linux, notify-send; on Windows, PowerShell toast notifications. None of these are required — Bee works silently if notifications aren't available.

Installing the plugin

Bee is a Claude Code plugin distributed through the Claude Code plugin marketplace. You add it through the plugin manager and then it is available in any Claude Code session. Once installed, commands are invoked as slash commands: */bee:init*, */bee:new-spec*, and so on. The *bee:* prefix is part of Claude Code's namespacing and means 'commands from the bee plugin'.

Initializing a project

Change directory into the project you want Bee to manage. Open Claude Code and run:

```
/bee:init
```

Bee will scan for manifest files (*package.json*, *composer.json*, *build.gradle.kts*), detect your stack(s), and ask you to confirm or adjust. For a single-stack project the interaction is short. For a monorepo with multiple apps, Bee will detect each one and ask about each.

After stack detection, Bee will ask about:

- Linter and test runner — detected from your manifests, confirmable.
- Implementation mode — economy (sonnet everywhere), quality (opus for critical work), or premium (opus everywhere). Default is premium.
- CI integration — whether you have GitHub Actions that Bee should be aware of.
- Context7 integration — whether to enable live docs lookups.
- Native notifications — whether to enable system notifications for long-running work.

When */bee:init* finishes, you have a populated *.bee/* directory. The *context-builder* agent will run automatically to scan your codebase and produce *CONTEXT.md* — a summary of your patterns that future agents will use as background knowledge.

Checking installation

Run */bee:progress*. If Bee is installed correctly, you will see a short status report with your stack, configuration, and (on a fresh project) a 'no active spec' line. If you see an error about missing *.bee/*, */bee:init* did not complete — re-run it.

Chapter 6

Your First Spec

We'll walk through a small but non-trivial feature: adding a 'saved items' list to a hypothetical e-commerce app. This is the kind of feature that has a backend model, an API endpoint, a UI surface, and a few edge cases — exactly the size where Bee's full workflow shines.

Start the discovery

```
/bee:new-spec "users can save products to a personal saved list"
```

Bee launches the *spec-shaper* agent. It will ask you questions like: *does the list persist across sessions? is it per-user or per-device? is there a maximum number of items? is order preserved? can an item be moved from the saved list to the cart? what happens if a saved product is deleted from the catalog? is there a saved-list page, a header count, both?*

Answer in plain English. If you don't know the answer, say so — Bee will flag the open question in the spec rather than invent an answer. If Bee asks something that is already obvious from your codebase, say so — the spec-shaper will read your code to verify.

The spec emerges

After enough rounds to reduce ambiguity (usually three to eight), the spec-shaper signals done and the *spec-writer* produces:

- *.bee/specs/0001-saved-items/spec.md* — the feature spec (behaviour only, no code).
- *.bee/specs/0001-saved-items/phases.md* — a phase breakdown, typically 3-6 phases.
- *.bee/specs/0001-saved-items/requirements.md* — the Q&A; record.
- *.bee/specs/0001-saved-items/ROADMAP.md* — a trace from each requirement to the phase(s) that cover it.

Next, *spec-reviewer* runs automatically. It is looking for missing success criteria, contradictions, gaps in the phase breakdown, and scope creep. If it flags issues, Bee loops back — you answer the follow-up questions and the spec is updated.

Read the spec

Before continuing, *read the spec*. This is not optional. The spec is the only thing between you and a feature built on Bee's interpretation of a five-minute conversation. Open *spec.md*, read it end-to-end, and if anything is wrong, run */bee:new-spec --amend* to fix it. Five minutes reading the spec saves an hour of rework later.

Chapter 7

Your First Phase

Plan the phase

```
/bee:plan-phase 1
```

Bee runs three agents in sequence. First, *researcher* scans your codebase for existing patterns: how do you structure models, how do you write migrations, what is your preferred pattern for API endpoints, where do authorization checks live. Second, *phase-planner* decomposes phase 1 into concrete tasks and groups them into waves. Third, *plan-reviewer* double-checks the plan against the spec.

Look at *.bee/specs/0001-saved-items/TASKS.md*. You will see a checklist like:

```
Phase 1 — Saved items model and persistence

Wave 1 (parallel):
  [ ] 1.1 Create migration for saved_items table
  [ ] 1.2 Create SavedItem model with user relation
  [ ] 1.3 Create factory and seeder

Wave 2 (depends on wave 1):
  [ ] 1.4 Add savedItems() relation to User model
  [ ] 1.5 Add repository methods for add/remove/list
```

Each task has its own research notes, test scenarios, and acceptance criteria. Skim them. If any task is missing or wrong, run */bee:plan-review 1* to re-review — you can also edit TASKS.md directly if you just want to tweak something.

Execute the phase

```
/bee:execute-phase 1
```

Bee spawns *implementer* agents for wave 1 tasks in parallel. Each one writes tests first, then implementation, then refactors. You can watch the output stream past — you'll see test files being created, tests running red, implementation changes, tests running green. When all wave 1 tasks are done, Bee runs the full test suite once, then moves to wave 2.

If any task fails (test stays red, lint errors, agent gets stuck), Bee's adaptive error recovery takes over: it analyses the failure, updates *LEARNINGS.md*, and retries with a different approach. If the failure persists, Bee stops and tells you — it will never ship half-working code just to keep moving.

Chapter 8

Your First Review and Commit

Review

```
/bee:review
```

Four agents scan the phase in parallel — *bug-detector*, *pattern-reviewer*, *plan-compliance-reviewer*, *stack-reviewer*. The *finding-validator* filters their output, and the survivors land in *REVIEW.md*.

Open *REVIEW.md*. Findings are grouped by severity (CRITICAL, HIGH, MEDIUM, LOW) and each one includes a file:line reference, the evidence, the impact, and a suggested fix. Read them all. For each one, decide: is this a real issue, is it stylistic, or is it a false positive I want to dismiss.

To auto-fix confirmed findings, run */bee:fix-implementation*. To mark a finding as a false positive, add it to *.bee/false-positives.md* with a short reason and the validator will ignore it on future reviews.

Test

```
/bee:test
```

Bee generates manual test scenarios based on the spec. For a saved-items phase 1, these will be basic model tests: 'a user can save an item', 'saving the same item twice is idempotent', 'deleting a user removes their saved items'. Walk through each one, verify it passes, and tell Bee to mark them verified.

Commit

```
/bee:commit
```

Bee shows the diff summary (files changed, lines added/removed, test pass/fail status) and proposes a commit message drawn from the spec and phase metadata. Review the diff one last time, approve the message, and the commit lands in git. That's phase 1 done.

Repeat for each phase. When all phases are committed, run */bee:complete-spec* for the full ceremony (final audit, tag, archive) or */bee:archive-spec* for a quick close.

Chapter 9

The .bee/ Directory Explained

The *.bee/* directory is the on-disk state that makes everything in the previous three chapters work. It looks scary the first time you open it — there are a lot of markdown files — but each file has a clear job and the structure is always the same. This chapter is the map.

Top-level files

File	Purpose	Edited by
config.json	Your Bee configuration (stacks, modes, review options)	init, profile
STATE.md	Current workflow state (spec, phase, last action, decisions)	Every conductor
PROJECT.md	Codebase index (structure, entry points, dependencies)	init, refresh-context
CONTEXT.md	Extracted patterns and conventions	context-builder
user.md	Your personal preferences (optional, user-authored)	You, via profile
LEARNINGS.md	Cross-phase insights the implementer should apply	Execute-phase
false-positives.md	Review findings you've dismissed	You, via review

Spec directories

Each spec lives in *.bee/specs/NNNN-slug/*. Inside, you'll find:

- *spec.md* — the feature spec (behaviour only, no code).
- *phases.md* — the phase breakdown.
- *requirements.md* — the Q&A; record from spec discovery.
- *ROADMAP.md* — requirements-to-phase traceability.
- *TASKS.md* — the task checklist with waves, research notes, acceptance criteria.
- *PLAN.md*, *PLAN-REVIEW.md* — plan-phase output and review.
- *REVIEW.md* — multi-agent review output.

Completed specs

When you complete a spec, it moves from *.bee/specs/* to *.bee/archive/*. This keeps the active-spec list clean while preserving history. Archived specs are still readable in the Bee Hive dashboard.

Quick tasks and seeds

.bee/quick/ holds quick-task plans created by */bee:quick*. *.bee/seeds/* holds ideas captured by */bee:seed*. *.bee/notes/* holds */bee:note* entries. *.bee/debug/* holds debug investigation sessions. *.bee/audit-specs/* holds spec drafts created by */bee:audit-to-spec*.

How to read STATE.md

STATE.md is the file you'll open most often. It has a fixed format: a header with the current spec and phase, a phase table showing status per phase, a decisions log with every consequential decision Bee has made on your behalf, and a 'last action' line with a suggestion for what to do next. When you come back from a break, reading STATE.md will orient you in under a minute.

Part III

Core Concepts

The ideas that shape how Bee runs

Chapter 10

Spec-Driven Development

We've used the term 'spec-driven' throughout. This chapter explains what that actually means in Bee, what it costs, and what it buys you.

The short definition

> *Spec-driven: the behaviour you want is written down, in human language, before any code is written, and every piece of code is traceable back to a line in that written behaviour.*

What Bee's specs look like

A Bee spec is a markdown file with a fixed shape:

- **Summary** — one paragraph naming the feature and its user value.
- **Behaviour** — numbered requirements, each one describing a user-visible behaviour in present-tense declarative sentences.
- **Success criteria** — measurable conditions for each requirement.
- **Out of scope** — explicit non-requirements, so later phases don't accidentally drift into them.
- **Open questions** — things that aren't decided yet.
- **Risks and constraints** — known gotchas.

What a spec does *not* contain: function names, class names, file paths, SQL, JSON schemas, or any other implementation detail. Those decisions live in the plan phase, not the spec. This separation is deliberate — it means the spec is stable even if you rewrite the whole implementation three times.

Phase breakdown

The spec is split into phases. A good phase is:

- Independently deliverable — you could ship phase 1 on its own and it would be coherent (even if incomplete).
- Testable in isolation — the phase has its own acceptance tests that don't depend on unshipped phases.
- Scope-bounded — small enough that a single plan-execute-review cycle can finish it without the conductor losing track.

Phases are ordered. Earlier phases can be built without later phases; later phases can assume earlier phases exist. This is why */bee:execute-phase 2* before phase 1 is committed will fail — the dependency invariant is violated.

The cost

Spec discovery takes 5-20 minutes per feature. For a one-hour task, that is a big overhead. For an all-week feature, it is nothing. The break-even is roughly a feature that would take longer than a half day to build without assistance. Below that, use */bee:quick*. Above that, pay the spec tax — it saves you more than it costs.

Chapter 11

TDD as Iron Law

Test-driven development is everywhere in Bee. It is worth being specific about what Bee means by TDD, because 'TDD' is a term that is used for several distinct practices.

Red-Green-Refactor

Bee's TDD is the strict Beck-style red-green-refactor cycle:

- **Red** — write a test that expresses the desired behaviour. Run the test. Confirm it fails, and that it fails for the right reason (compilation error is not good enough — the assertion must run and fail).
- **Green** — write the minimum code that makes the test pass. Nothing more. Nothing clever. Nothing generic. Just enough code that the test is green.
- **Refactor** — clean up the code while tests remain green. Rename, extract, inline, simplify. Run tests after every meaningful refactor.

The cycle runs task by task, not feature by feature. An implementer handling task 1.3 will do red-green-refactor for 1.3 only, not for the whole phase. This is what makes parallel wave execution possible: each implementer owns a scoped test set and a scoped code change.

Why strict TDD

Strict TDD in Bee exists for one reason: *evidence*. At the end of a task, Bee needs to be able to point at the test and say 'this test did not exist before, it was written, it failed, then it passed'. That sequence is the proof that something changed in the right direction. Without it, you have faith — and Bee refuses to run on faith.

The *SubagentStop* validator for implementers checks for this sequence. An implementer that writes implementation first and tests afterwards (or, worse, writes passing tests against existing code) will have its output rejected and will be asked to redo the task following the cycle correctly.

When TDD doesn't fit

There are a few cases where strict TDD is impractical: one-off scripts, exploratory spikes, UI polish. Bee offers escape hatches:

- */bee:quick --fast* bypasses the TDD cycle for genuinely trivial edits.
- Spike work is done outside Bee entirely; Bee is for things you intend to keep.
- UI polish that is not reachable by unit tests is verified with */bee:test-e2e* or by manual test scenarios through */bee:test*.

These are exceptions. For every other kind of work, Bee's TDD is non-negotiable, and that is a feature, not a limitation.

Chapter 12

Wave-Based Parallel Execution

Bee runs multiple agents in parallel whenever it can. The mechanism is wave-based execution, and it is worth understanding in detail because it has implications for how you plan phases.

What a wave is

A wave is a set of tasks that can run at the same time without interfering with each other. Tasks in wave 1 have no dependencies. Tasks in wave 2 may depend on wave 1 tasks but not on each other. Tasks in wave 3 may depend on wave 2 (and transitively wave 1) but not on each other. And so on.

The *phase-planner* agent computes the wave assignment during */bee:plan-phase*. It looks at each task's dependencies — both explicit (stated in the plan) and implicit (shared files, shared modules) — and produces a partial order. Tasks with the same depth in the partial order become the same wave.

Why waves matter

Sequential execution of a 10-task phase takes 10 times as long as parallel execution of the same 10 tasks (give or take the cost of spawning agents). On a modern model and a well-planned phase, that is the difference between a 45-minute execution and a four-minute execution. For large features with tens of phases, wave execution turns a day's work into an hour.

The scoped testing trick

Parallel execution sounds dangerous — what if four implementers all touch the same file? Bee has two defences:

- The phase-planner avoids scheduling tasks that touch the same file in the same wave. Conflicting writes do not happen.
- Each implementer runs *only* the tests that cover the files it owns, not the full suite. This is the scoped testing rule. It means parallel agents do not have to wait on each other's tests, and it cuts the runtime of a wave by 70% or more on suites with slow tests.

The conductor compensates for scoped testing by running the full suite once at the end of each wave. If something cross-cuts and breaks, it surfaces at the wave boundary and not at phase-end, which keeps the blast radius small.

What to plan for

Phases that parallelize well have lots of independent files (many model files, many component files, many endpoint files). Phases that parallelize poorly have one central file that every task has to modify. If you find yourself with a phase where every task touches the same file, that is usually a sign that the phase is too small — merge it into the next phase and re-plan.

Chapter 13

Disk is Truth: State Management

The disk-is-truth principle is simple to state but has deep consequences. Everything Bee does is durable because it is written to a file before the command considers it done.

What lives on disk

Everything that affects future behaviour. Specs. Phases. Tasks. Decisions. Learnings. Review findings. Audit results. Debug sessions. User preferences. The list of false positives. The implementation mode. The active phase number. The last action taken. The timestamp of the last run.

What does not live on disk

Transient conversation, output streams that are not yet decisions, intermediate thinking. These are consumed and discarded. If something matters after the next session, it is written.

Why this matters practically

Sessions crash. Context windows fill. Networks hiccup. Laptops suspend. Teams hand off work. The question 'what is the state of this feature' has exactly one correct answer, and that answer lives in *.bee/*. You can take a week off, come back, read STATE.md, and know precisely where you are. You can hand the project to a teammate and they can pick it up without a verbal briefing. You can run */bee:resume* after a crash and Bee will reload the full context from files.

The rule of thumb

> *If it's not in .bee/, it didn't happen.*

Chapter 14

Multi-Agent Orchestration

Bee's commands spawn multiple agents by design. Understanding why — and how Bee keeps them from stepping on each other — makes the review and audit commands feel less like magic and more like a disciplined technique.

The single-agent problem

One agent reviewing a code change is cheap but narrow. It will find some bugs; it will miss others. It will overlook the things its training data underrepresents. Adding more agents of the same kind doesn't help — they all have the same blind spots.

The specialist solution

Bee's multi-agent reviews use *different specialists* in parallel, each one trained — through its system prompt and its attached skill — to look at a different dimension. The code review uses:

- *bug-detector* — logic errors, off-by-ones, security issues, error handling gaps.
- *pattern-reviewer* — deviations from project conventions (naming, layering, file placement, error-handling style).
- *plan-compliance-reviewer* — does the code actually do what the plan said it would, and does it cover all the requirements.
- *stack-reviewer* — framework best practices, anti-patterns, idiomatic usage, versioning.

Because each agent has a narrow focus, each one digs deep in its own dimension. Combined, they cover more ground than any generalist agent could.

The validator solution

Multiple specialists generate more findings but also more false positives. Bee's answer is a *finding-validator* agent that runs after the specialists and re-reads the code for each finding. Its job is triage: *confirmed bug*, *stylistic preference*, *false positive*, *needs more context*. Only confirmed findings go into REVIEW.md. This is why Bee's reviews are useful without being noisy.

The consolidator solution

For deep audits with many parallel agents (*/bee:audit* runs 9 specialists), the *audit-report-generator* merges findings across agents, deduplicates overlapping findings, and writes a single AUDIT-REPORT.md. The same agent assigns severity using a shared calibration table so 'HIGH' means the same thing whether it comes from the security or the performance auditor.

Chapter 15

Implementation Modes and Model Tiering

Bee runs on multiple models depending on the job. The choice is controlled by the *implementation_mode* configuration value, which has three settings.

The three modes

Mode	Scanning / planning agents	Critical / implementation agents
economy	Sonnet	Sonnet
quality	Sonnet	Opus
premium	Opus	Opus

Scanning and planning agents include *researcher*, *phase-planner*, *spec-writer*, and most of the audit agents. Critical agents include *implementer*, the four review specialists, *fixer*, and *finding-validator*.

How to choose

The right mode depends on what you value most right now:

- **economy** — when you want speed and cost-efficiency on well-understood work. Spec discovery and planning are still fine on Sonnet; it's the implementation detail where you may notice shallower choices.
- **quality** — the default recommendation. Planning on Sonnet is cheap and fast; implementation and review on Opus is where you want the better reasoning.
- **premium** — when you are working on a critical feature and you want the best reasoning Bee can give you on every step, including research and spec writing.

You can change mode per session by updating *config.implementation_mode* directly or by toggling it through the Bee Hive dashboard. You can also set it during */bee:new-spec* if you want this one spec to run at a different tier.

Part IV

Complete Command Reference

Every command, what it does, when to use it, and how

Chapter 16

Setup and Navigation

These commands are the scaffolding of a Bee session: starting a project, finding out where you are, jumping back in after a break, and asking Bee to point you at what to do next. You will use the navigation commands more than any other group — they are the friction between Bee's deep state and your short attention span.

`/bee:init`

One-time project setup

Detects your stack(s), creates the *.bee/* directory, generates the initial configuration, extracts codebase context, and installs the Bee statusline. Running it on an already-initialized project performs a safe re-initialization: config is updated, but specs, state, and user preferences are preserved.

When to use

- The first time you use Bee in a project.
- After switching Bee versions, to refresh the statusline and pick up new configuration defaults.
- After restructuring a monorepo, to re-detect stacks in new subdirectories.

What it writes

- *.bee/config.json* — stack list, linter, test runner, implementation mode, CI integration.
- *.bee/STATE.md* — initial workflow state (NO_SPEC).
- *.bee/PROJECT.md* — codebase index.
- *.bee/CONTEXT.md* — extracted patterns (via context-builder).
- Optional: *CLAUDE.md*, *.gitignore* entries.

Examples

```
/bee:init
```

Gotchas

- If you have multiple apps in one repository, Bee will detect each independently — confirm every entry.
- On re-init, STATE.md and specs are never touched. Bee goes to great lengths to make */bee:init* idempotent.
- The context-builder run at the end can take a minute or two on larger codebases. Let it finish.

Related

`/bee:update`, `/bee:refresh-context`, `/bee:progress`

/bee:progress

Show current project state

Renders a concise report of the current spec, phase status, last action taken, recent decisions, and a suggestion for what to do next. It reads STATE.md and formats it for humans instead of agents.

When to use

- Opening a session to remind yourself where things are.
- Deciding what to run next when several things are in flight.
- Verifying that a previous command actually updated state.

What it writes

- Read-only. Writes nothing.

Examples

```
/bee:progress
```

Related

`/bee:next`, `/bee:resume`, `/bee:eod`

/bee:next

Suggest the next command

Reads STATE.md and recommends exactly one command to run next based on where the workflow is. If a phase is planned but not executed, it suggests */bee:execute-phase*. If executed but not reviewed, */bee:review*. And so on down the lifecycle.

When to use

- You forgot what comes next and don't want to re-read the lifecycle.
- You have multiple branches in flight and want Bee to pick.
- You want a second opinion on whether to keep going or stop.

Examples

```
/bee:next
```

Related

`/bee:progress`, `/bee:do`

/bee:do

Natural-language intent router

You describe what you want in plain English ("let me plan the next phase", "run a full audit", "just fix this one button") and Bee translates that intent into the right command with the right arguments. It is the most forgiving entry point to the whole system.

Arguments

`description` — Free-text description of what you want to do

When to use

- You know what you want to do but not which command does it.
- Onboarding someone new who hasn't memorized the command list.
- You want to stay in English instead of in a command syntax.

Examples

```
/bee:do plan the next phase
```

```
/bee:do audit the whole codebase for security issues
```

```
/bee:do quickly fix the button alignment on the login page
```

Related

`/bee:next`, `/bee:help`

/bee:resume

Restore full context after a break

Reads STATE.md, CONTEXT.md, the current spec, current phase, and any in-progress debug or forensic sessions, and rebuilds the working context so you can continue without re-explaining anything.

When to use

- Starting a new Claude Code session on an existing Bee project.
- After a crash, power loss, or network hiccup.
- Coming back from a long break.

Examples

```
/bee:resume
```

Related

`/bee:pause`, `/bee:progress`

/bee:pause

Save work-in-progress for later

Captures the current in-memory state into STATE.md along with a "paused here" marker, so the next */bee:resume* can pick up exactly where you left off.

When to use

- End of the day on an unfinished phase.
- Before switching to an urgent unrelated task.
- Before running */bee:compact* if you want a clean restart point.

Examples

```
/bee:pause
```

Related

`/bee:resume`, `/bee:compact`, `/bee:eod`

/bee:compact

Smart context compaction

Preserves Bee's workflow state in *.bee/COMPACT-CONTEXT.md*, then runs Claude Code's */compact* to shrink the conversation. The next command call rebuilds context from the preserved file, so you lose conversation noise but keep workflow state.

When to use

- Context is getting long and you want to continue in the same session without losing Bee's understanding of where you are.
- Before a large execute-phase run that you know will add lots of output.

Examples

```
/bee:compact
```

Gotchas

- Compact is a form of forgetting. If you were mid-discussion about an open spec question, that will be lost — only the state on disk survives.

Related

`/bee:resume`, `/bee:pause`

`/bee:profile`

View or edit user profile and preferences

Opens the *.bee/user.md* profile for editing. Your profile holds preferences the agents should respect: tone, testing style, review strictness, preferred naming conventions, things to avoid. Preferences are injected into every subagent via the *SubagentStart* hook.

When to use

- First time setup to document your preferences.
- After noticing Bee repeatedly making a choice you disagree with — codify the rule here instead of correcting it every time.

Examples

```
/bee:profile
```

Related

`/bee:memory`

`/bee:memory`

View accumulated agent memories

Lists the project-level memories agents have accumulated — LEARNINGS, false positives, cross-session knowledge threads — and lets you prune or edit them.

When to use

- You suspect Bee is carrying a stale belief about your project.
- You want to see what Bee has learned from previous phases.

Examples

```
/bee:memory
```

Related

`/bee:profile`, `/bee:thread`

/bee:refresh-context

Rebuild CONTEXT.md from scratch

Re-runs the *context-builder* agent over the whole project to produce a fresh CONTEXT.md. Useful after large refactors that invalidate prior pattern detection.

When to use

- You just merged a big restructure and the previous CONTEXT.md describes a world that no longer exists.
- You added a new subproject or new layer that Bee should be aware of.

Examples

```
/bee:refresh-context
```

Gotchas

- This rewrites CONTEXT.md — if you'd hand-edited it, save a copy first.

Related

`/bee:init`

/bee:update

Update Bee statusline and cleanup

Installs the latest Bee honeycomb statusline globally and removes any legacy per-project copies. Run once after updating the Bee plugin.

When to use

- After updating the Bee plugin to a new version.

Examples

```
/bee:update
```

Related

`/bee:init`

/bee:help

Full command reference and usage guide

Prints the full list of commands grouped by purpose, with one-line descriptions. If you know the command group you want, pair */bee:help* with grep; otherwise, use */bee:do*.

When to use

- When you need a quick in-session reference.

Examples

```
/bee:help
```

Related

`/bee:do`, `/bee:next`

Chapter 17

Specification Commands

Specification is the front of the Bee lifecycle. This group covers the commands that take an idea in your head and produce a reviewed spec + phase breakdown on disk. Get this phase right and the rest of the lifecycle is downhill; get it wrong and every subsequent command will be building on the wrong foundation.

`/bee:new-spec`

Create or amend a feature spec

Launches the *spec-shaper* agent for adaptive discovery — a conversational back-and-forth that asks whatever questions are needed to reduce ambiguity, however many rounds it takes. When discovery is complete, the *spec-writer* produces *spec.md*, *phases.md*, *requirements.md*, and *ROADMAP.md*. Finally *spec-reviewer* audits the result and loops back if it finds gaps.

Arguments

`description` — Optional free-text description of the feature

Flags

`--amend` — Modify the current spec instead of creating a new one

`--from-discussion PATH` — Start from an existing discussion or audit-spec file

When to use

- Starting a new feature that is worth more than 30 minutes of work.
- Turning an audit finding into an actionable spec (use *--from-discussion*).
- Amending an existing spec (use *--amend*).

What it writes

- *.bee/specs/NNNN-slug/spec.md*
- *.bee/specs/NNNN-slug/phases.md*
- *.bee/specs/NNNN-slug/requirements.md*
- *.bee/specs/NNNN-slug/ROADMAP.md*

Examples

```
/bee:new-spec
```

```
/bee:new-spec "users can save products to a wishlist"
```

```
/bee:new-spec --amend
```

```
/bee:new-spec --from-discussion .bee/audit-specs/critical-auth.md
```

Gotchas

- Resist the urge to answer every question quickly. The spec-shaper rewards precision. A half-answered question becomes a half-specced feature.
- If the spec-shaper asks you something obvious from your code, say so — it will read the code.

Related

`/bee:discuss`, `/bee:plan-phase`, `/bee:ui-spec`

`/bee:discuss`

Brainstorming before a spec

Opens a *discuss-partner* session that is more open-ended than */bee:new-spec*. Useful when you're not yet ready to commit to a spec and want to explore ideas against the actual codebase. Notes are written to *.bee/discussions/NNNN-topic.md* and can later seed a spec with */bee:new-spec --from-discussion*.

Arguments

`description` — Optional topic to discuss

When to use

- You have an idea but aren't sure whether it's worth a full spec yet.
- You want the AI to sanity-check feasibility before committing.
- You are exploring several alternatives and want to pick one.

Examples

```
/bee:discuss should we migrate from REST to tRPC
```

```
/bee:discuss
```

Related

`/bee:new-spec`, `/bee:seed`

/bee:plan-phase

Plan a single phase

Runs the planning pipeline for one phase: researcher (scan codebase for patterns), phase-planner (decompose into tasks and waves), plan-reviewer (sanity check against the spec). Produces a complete TASKS.md for the phase.

Arguments

`phase-number` — The phase number to plan (e.g. 1, 2, 3)

When to use

- After */bee:new-spec*, before executing phase 1.
- Between phases during a feature build.
- When re-planning a phase after a spec amendment.

What it writes

- *TASKS.md* updated with task list + waves for the phase.
- *PLAN.md* with research notes and dependency rationale.
- *PLAN-REVIEW.md* with the plan-reviewer's findings.

Examples

```
/bee:plan-phase 1
```

```
/bee:plan-phase 3
```

Related

`/bee:plan-review`, `/bee:plan-all`, `/bee:execute-phase`

/bee:plan-review

Standalone plan review

Re-runs the plan-reviewer on an existing planned phase. Useful if you've hand-edited TASKS.md and want Bee to double-check, or if the spec was amended after planning and you want to catch drift.

Arguments

`phase-number` — The phase to re-review

When to use

- After editing TASKS.md manually.
- After a spec amendment that might invalidate a planned phase.
- Before */bee:execute-phase* when you want an extra sanity check.

Examples

```
/bee:plan-review 2
```

Related

`/bee:plan-phase`

/bee:plan-all

Plan all remaining phases autonomously

Runs */bee:plan-phase* for every unplanned phase in sequence, with autonomous review per phase, then runs a cross-plan consistency review across all planned phases to catch inter-phase contracts and dependency chains.

When to use

- Preparing for a */bee:ship* run — you want everything planned and reviewed up front, then execution to proceed unattended.
- Large features where per-phase planning is too slow.

Examples

```
/bee:plan-all
```

Gotchas

- Because later phases depend on earlier ones, plan-all may make decisions that change when earlier phases are actually executed. The cross-plan review catches most of this, but not all.

Related

`/bee:plan-phase`, `/bee:ship`

/bee:add-phase

Append a new phase to the current spec

Adds a new phase to the end of the current spec. Useful when you realize mid-feature that there's a slice of work you want to capture as a distinct phase rather than wedging it into an existing one.

Arguments

`phase-description` — Optional description of the new phase

When to use

- Mid-feature discovery of a new scope slice.
- Splitting an oversized phase into two.

Examples

```
/bee:add-phase "audit logging for saved-item actions"
```

Related

`/bee:insert-phase`, `/bee:new-spec`

/bee:insert-phase

Insert an urgent phase mid-spec

Inserts a new phase at a decimal position (e.g. 2.1) between existing phases, so urgent work can be scheduled without renumbering. The new phase is planned from scratch.

Arguments

`position` — Decimal position, e.g. 2.1, 2.2, 3.1

When to use

- You discover a dependency that must be built before the next normal phase.
- Urgent hotfix work that belongs inside the current spec.

Examples

```
/bee:insert-phase 2.1
```

Related

`/bee:add-phase`

/bee:ui-spec

UI design contract for a frontend phase

Generates a UI design contract — a written specification of the visual and interaction design for a phase that ships UI. It covers layout, states, breakpoints, copy, accessibility, and micro-interactions. Becomes an input to the frontend implementer and the ui-reviewer.

When to use

- Starting a phase that ships visible UI.
- Before */bee:execute-phase* for any frontend-heavy work.

Examples

```
/bee:ui-spec
```

Related

`/bee:new-spec`, `/bee:ui-review`

Chapter 18

Execution Commands

Execution is where code is actually written. This group is small but powerful: four commands that cover everything from the most disciplined per-phase TDD run to a fully autonomous end-to-end ship.

`/bee:execute-phase`

Execute a planned phase with wave-based parallel TDD

Reads TASKS.md for the given phase, spawns implementer agents in parallel wave by wave, and runs each task through the red-green-refactor cycle. Scoped testing keeps parallel agents from stepping on each other; the conductor runs the full test suite once per wave boundary. Adaptive error recovery retries failed tasks with alternative approaches and escalates after repeated failure.

Arguments

`phase-number` — The phase to execute

When to use

- After */bee:plan-phase N* produces a reviewed plan.
- Resuming an interrupted phase (execution is resumable via STATE.md).

What it writes

- Production code and test changes in your working directory.
- Task checkmarks in TASKS.md.
- Decisions and learnings in STATE.md and LEARNINGS.md.

Examples

```
/bee:execute-phase 1
```

Gotchas

- You cannot execute phase N+1 before phase N is committed (Bee enforces this via the phase-status state machine). Override only by editing STATE.md directly, and only if you know what you're doing.
- If your test suite is slow, scoped testing still helps, but the wave-end full run will dominate. Consider parallel test runners.

Related

`/bee:plan-phase`, `/bee:review`, `/bee:ship`

`/bee:quick`

Fast-track task without a full spec

For work that is not worth the full spec pipeline. Quick has three modes: default is TDD-enforced with researcher + implementer, *--fast* skips agents and runs directly (for truly trivial edits), and *--review* adds a 4-agent review before the commit. Each quick task is saved to *.bee/quick/* so it can be re-run or amended later.

Arguments

`description` — Free-text description of the task

Flags

`--fast` — Direct execution, no agents, bypasses TDD

`--amend N` — Modify an existing quick task by number

`--review` — Run 4-agent review before commit

When to use

- Work that takes less than 30 minutes end-to-end.
- Bug fixes where the spec-plan overhead isn't worth it.
- Small UI tweaks, copy changes, dependency bumps.

Examples

```
/bee:quick fix the login button alignment
```

```
/bee:quick --fast update the footer copyright year
```

```
/bee:quick --review refactor auth middleware error handling
```

```
/bee:quick --amend 3
```

Gotchas

- *--fast* skips TDD. This is an escape hatch. Use it sparingly and only for edits you would not hesitate to ship without tests.
- Quick tasks don't have phases. For anything big enough to need phases, use */bee:new-spec* instead.

Related

`/bee:new-spec`, `/bee:execute-phase`

`/bee:autonomous`

Plan + execute + review per phase

Runs the full per-phase pipeline (plan, review plan, execute, review code, test) autonomously, one phase at a time, with a fresh context between phases. Does *not* commit — you still review the final result and commit manually. The *--from* and *--to* flags let you run a subset of phases.

Flags

`--from N` — Start at phase N

`--to N` — Stop after phase N

`--skip-discuss` — Skip the discussion prompts

When to use

- You trust your spec and want to minimize your own time in the loop.
- You want autonomous behaviour but still want a fresh context between phases so earlier-phase noise doesn't affect later work.

Examples

```
/bee:autonomous
```

```
/bee:autonomous --from 2 --to 4
```

Related

`/bee:ship`, `/bee:execute-phase`

`/bee:ship`

Execute all plan-reviewed phases fully autonomously

The top-of-the-stack autonomous pipeline. Assumes all phases are already plan-reviewed. Executes every phase in sequence, runs the review loop after each, does the final implementation review at the end, and logs every decision to STATE.md. Zero interaction: Bee will not ask you anything, it will not stop for approval, it will continue optimistically after review iterations are exhausted. Resumable after crashes.

Flags

`--skip-discuss` — Skip discussion prompts entirely

When to use

- After */bee:plan-all* on a well-specified feature.
- When you want to walk away for an hour and return to a shipped-but-uncommitted feature.

Examples

```
/bee:ship
```

Gotchas

- Still does not auto-commit. You return to a finished set of changes that you review and commit manually.
- If the spec has ambiguity the plan-review missed, ship may make decisions you disagree with. Every decision is logged in STATE.md — review the log.

Related

`/bee:plan-all`, `/bee:autonomous`

Chapter 19

Quality Commands

Quality commands are how Bee turns "code that passes tests" into "code you would merge". They are built around the multi-agent-plus-validator pattern from Chapter 14: parallel specialists find lots of potential issues, a validator filters out the noise, you get a focused list of things to actually fix.

`/bee:review`

Multi-agent parallel code review

Runs four specialist agents in parallel over the current phase: *bug-detector*, *pattern-reviewer*, *plan-compliance-reviewer*, and *stack-reviewer*. The *finding-validator* then re-reads the code for each finding and classifies it as confirmed, stylistic, or false positive. Survivors are written to REVIEW.md grouped by severity. If the review loop is enabled, Bee will auto-fix and re-review.

Flags

`--phase N` — Review a specific phase instead of the current one

`--loop` — Enable auto-fix + re-review loop (can also be on by config)

When to use

- After every */bee:execute-phase*.
- Before */bee:commit* on meaningful changes.
- Any time you want a second set of eyes on a change.

What it writes

- *.bee/specs/.../REVIEW.md* — findings grouped by severity with file:line references and suggested fixes.

Examples

```
/bee:review
/bee:review --phase 2
/bee:review --loop
```

Related

`/bee:fix-implementation`, `/bee:swarm-review`

/bee:swarm-review

Deep segmented parallel review

A heavier review mode that splits the phase into segments and runs the specialists against each segment in parallel, then consolidates through *swarm-consolidator*. Use this when a phase is large enough that a normal review risks missing things in the corners.

When to use

- Phases with many files (>20) or many concerns.
- Critical feature final review.
- When */bee:review* produces suspiciously few findings on a large change.

Examples

```
/bee:swarm-review
```

Related

`/bee:review`, `/bee:review-implementation`

/bee:review-implementation

Cross-phase spec compliance review

Runs a final review that considers the whole spec, not a single phase. In spec mode, 5 agents per stack (including audit-bug-detector) scan for cross-phase contract mismatches and spec compliance gaps. In ad-hoc mode (no active spec), 3 agents scan the current working changes.

When to use

- Before */bee:complete-spec* or */bee:archive-spec*.
- After */bee:ship* to verify cross-phase integrity.
- When debugging an integration bug that spans phase boundaries.

Examples

```
/bee:review-implementation
```

Related

`/bee:review`, `/bee:audit`, `/bee:complete-spec`

/bee:fix-implementation

Apply fixes from a review report

Reads REVIEW.md (or a provided review file), spawns *fixer* agents in parallel across files (sequential within the same file), and applies the fixes. After fixing, the phase can be re-reviewed to confirm the issues are gone.

Arguments

`path/to/REVIEW.md` — Optional path to a specific review file

When to use

- After */bee:review* produces confirmed findings you accept.
- After */bee:swarm-review* or */bee:review-implementation*.

Examples

```
/bee:fix-implementation
```

```
/bee:fix-implementation .bee/specs/0003/REVIEW.md
```

Gotchas

- The fixer is conservative — it addresses one finding at a time and stops if it can't cleanly fix something without making things worse. Unfixed findings stay in REVIEW.md.

Related

`/bee:review`, `/bee:swarm-review`

/bee:ui-review

Six-pillar UI quality audit

Runs the *ui-auditor* against the current UI work using a six-pillar rubric: layout, typography, colour and contrast, responsiveness, interaction states, and accessibility. Produces a structured UI audit report with findings per pillar.

When to use

- After any phase that ships visible UI.
- Before shipping UI-heavy features.

Examples

```
/bee:ui-review
```

Related

`/bee:ui-spec`, `/bee:review`

Chapter 20

Testing Commands

TDD covers the unit-test layer automatically during execution. This group covers the other layers: manual test scenarios, requirement-driven test generation from acceptance criteria, and Playwright E2E tests for UI work.

`/bee:test`

Generate manual test scenarios

Generates human-runnable test scenarios for the current phase and walks you through each one. Unlike */bee:test-gen*, these are intended to be executed by a person (or a QA partner), typically for UI or cross-system behaviours that unit tests don't cover well.

When to use

- After */bee:review*, before */bee:commit*.
- As part of EOD sanity-checking.

Examples

```
/bee:test
```

Related

`/bee:test-gen`, `/bee:test-e2e`

`/bee:test-gen`

Requirement-driven test generation

Reads acceptance criteria from the spec and produces matching automated tests. Useful for filling in coverage gaps where the initial TDD pass focused on happy paths and missed edge cases.

When to use

- After */bee:execute-phase* if you notice coverage gaps in edge cases.
- For existing code that needs retroactive tests before refactoring.

Examples

```
/bee:test-gen
```

Related

`/bee:test`, `/bee:test-e2e`

/bee:test-e2e

Playwright E2E tests with POM

Generates Playwright end-to-end tests structured with the Page Object Model. Uses the bundled Playwright skill for selectors, fixtures, network mocking, and authentication patterns. Optionally runs the generated tests in-place.

Arguments

`description` — Optional test description

Flags

`--run` — Run the generated tests after creation

When to use

- After any frontend-heavy phase where user flows cross multiple pages.
- Regression protection for critical flows.

Examples

```
/bee:test-e2e
```

```
/bee:test-e2e "checkout flow with coupon" --run
```

Related

`/bee:test`, `/bee:ui-review`

Chapter 21

Audit Commands

Audits are deep, read-only, multi-agent scans of the entire codebase. Unlike reviews (which target a phase or diff), audits target everything, using specialized auditor agents that focus on security, databases, APIs, architecture, performance, and more. Audits are the backbone of the 'inherited project takeover' workflow.

`/bee:audit`

Comprehensive multi-agent code audit

Runs up to 10 auditor agents in parallel over the whole codebase: *security-auditor*, *error-handling-auditor*, *database-auditor*, *architecture-auditor*, *api-auditor*, *frontend-auditor*, *performance-auditor*, *testing-auditor*, *audit-bug-detector*, and the validator. Findings are consolidated into AUDIT-REPORT.md (human-readable) and audit-findings.json (machine-readable for */bee:audit-to-spec*).

Flags

`--only security,database,...` — Run only the listed auditor agents

`--skip-validation` — Skip the validator stage (faster, noisier)

`--severity critical,high` — Filter findings by severity at output time

When to use

- Inheriting a codebase you didn't write.
- Periodic health check on a long-running project.
- Before a big refactor, to know what you're starting from.
- Before a release, as a pre-flight check.

What it writes

- *.bee/AUDIT-REPORT.md* — human-readable findings grouped by severity and agent.
- *.bee/audit-findings.json* — machine-readable findings for downstream tooling.

Examples

```
/bee:audit
/bee:audit --only security,database
/bee:audit --severity critical,high
```

Gotchas

- A full audit is expensive (time and tokens). Use *--only* to scope it when you know which dimensions matter.
- Skipping validation can surface false positives. Useful for rapid triage, not for final reports.

Related

`/bee:audit-to-spec`, `/bee:audit-spec`, `/bee:review-implementation`

/bee:audit-to-spec

Convert audit findings into actionable specs

Reads audit-findings.json and generates spec drafts in *.bee/audit-specs/*. Findings are grouped by severity: CRITICAL findings become individual spec drafts, HIGH findings are grouped, MEDIUM are consolidated into cleanup drafts, LOW are collected into a single 'nice to have' draft. Each draft can then feed */bee:new-spec --from-discussion*.

Flags

`--critical` — Only generate specs for CRITICAL findings

`--high` — Only generate specs for HIGH findings

`--all` — Generate specs for all severity levels

`--dry-run` — Preview what would be created without writing files

When to use

- Immediately after */bee:audit* on an inherited project.
- When the audit report is too long to address by hand and you need structured follow-up.

Examples

```
/bee:audit-to-spec
```

```
/bee:audit-to-spec --critical
```

```
/bee:audit-to-spec --dry-run
```

Related

`/bee:audit`, `/bee:new-spec`

/bee:audit-spec

Spec traceability matrix

Builds a traceability matrix from the spec's requirements to the code that implements them, and highlights requirements with no implementation trace. Useful for catching drops between spec and implementation on long specs.

When to use

- After */bee:review-implementation* for extra assurance.
- Before */bee:complete-spec*.
- When auditing a spec that was implemented in multiple sessions.

Examples

```
/bee:audit-spec
```

Related

`/bee:review-implementation`, `/bee:complete-spec`

Chapter 22

Debug and Recovery Commands

When things go wrong — a bug you can't find, a workflow that derailed, a project that feels unhealthy — this group gets you back on track. All three commands are investigative: they read, hypothesize, and report. None of them modify your code.

`/bee:debug`

Systematic bug investigation

Launches a *debug-investigator* session with structured hypothesis tracking. You describe the bug, Bee generates hypotheses ranked by likelihood, investigates each one against the code, and tracks which have been ruled out. Sessions are saved to *.bee/debug/NNNN-slug/* and can be resumed.

Flags

`--resume SESSION_ID` — Resume a previous debug session

When to use

- A bug you've already tried to debug for more than 10 minutes.
- Intermittent bugs where you need to reason about race conditions or environmental factors.
- Cross-layer bugs where the symptom is in one place and the cause in another.

What it writes

- *.bee/debug/NNNN-slug/INVESTIGATION.md* — hypothesis list, evidence, and current conclusions.

Examples

```
/bee:debug
```

```
/bee:debug --resume 0003
```

Related

`/bee:forensics`, `/bee:health`

/bee:forensics

Read-only workflow forensics

Diagnoses what went wrong in a past workflow execution: phases that silently skipped, reviews that were never run, tests that got marked verified without evidence, decisions that deviated from the spec. Uses a 4-factor severity escalation rubric to rank findings. Read-only — produces a forensic report but does not fix.

When to use

- Something shipped that shouldn't have.
- You suspect a previous */bee:ship* run made questionable decisions and want a forensic check.
- You inherited the project and want to understand its workflow history.

Examples

```
/bee:forensics
```

Related

`/bee:debug`, `/bee:health`, `/bee:audit`

/bee:health

13-check project health diagnostics

Runs 13 diagnostic checks against the project and Bee state: STATE consistency, test suite health, lint cleanliness, dependency drift, spec hygiene, review coverage, and more. Compares with previous health runs to detect trends (improving, stable, degrading).

When to use

- Weekly or monthly project health check.
- Before taking a break from a project, to capture a baseline.
- After returning from a break, to see what's changed.
- As part of EOD.

What it writes

- *.bee/HEALTH-REPORT.md* — per-check status, trend arrows, and overall health score.

Examples

```
/bee:health
```

Related

`/bee:eod`, `/bee:forensics`

Chapter 23

Session, Ideas, and Backlog Commands

This group covers the supporting cast: commits, end-of-day ceremony, idea capture, note-taking, thread persistence, and parallel worktrees. They are small commands but they are the ones that make Bee feel like a workflow you actually inhabit rather than a one-shot tool.

`/bee:commit`

Human-approved commit with diff summary

Shows a diff summary, proposes a commit message drawn from the current spec and phase metadata, and asks for your confirmation. On approval, runs git commit with the chosen message. Never commits without explicit approval.

When to use

- After */bee:review* and */bee:test*, at the end of each phase.
- At the end of any quick task.

Examples

```
/bee:commit
```

Gotchas

- If you want to edit the proposed message, do it before confirming. Bee will use whatever you confirm.
- Commit is deliberately a separate step from the rest of the pipeline. Bee will never sneak one in during */bee:ship*.

Related

`/bee:review`, `/bee:complete-spec`

/bee:complete-spec

Full spec completion ceremony

The full close-out: runs a final audit of the spec, tags the commit, archives the spec directory into *.bee/archive/*, resets STATE.md, and writes a completion summary. Use this for the close of a meaningful feature.

When to use

- All phases of a spec are committed and you are ready to call it done.

Examples

```
/bee:complete-spec
```

Related

`/bee:archive-spec`, `/bee:review-implementation`

/bee:archive-spec

Quick archive without ceremony

A lightweight alternative to */bee:complete-spec*: moves the spec to archive and resets STATE.md without the final audit or tagging. Use when you are confident the spec is done and want a fast close-out.

When to use

- Small specs where a full completion ceremony is overkill.
- Closing out old specs you want to clean up but not re-audit.

Examples

```
/bee:archive-spec
```

Related

`/bee:complete-spec`

/bee:eod

End-of-day integrity check

Runs 4 parallel EOD audits: *test-auditor* (test suite health), *integrity-auditor* (STATE matches disk reality), *testing-auditor* (coverage gaps), plus a velocity snapshot. Produces a short EOD summary with things to fix tomorrow and anything that looks unhealthy.

When to use

- End of the working day on an active project.
- Before any long break.

Examples

```
/bee:eod
```

Related

`/bee:health`, `/bee:pause`

`/bee:seed`

Capture an idea with trigger conditions

Saves an idea to *.bee/seeds/* with trigger conditions — 'when we migrate to Postgres', 'when traffic hits 10k/day', 'after we ship v2'. Seeds are lightweight future-intent records that don't distract from current work but aren't forgotten either.

Arguments

`idea` — Free-text description of the idea

When to use

- A thought that isn't worth stopping to spec right now but is worth not losing.
- Ideas that depend on future conditions.

Examples

```
/bee:seed "migrate from MySQL to Postgres when we need JSONB"
```

```
/bee:seed "add WebAuthn after we ship the new settings page"
```

Related

`/bee:backlog`, `/bee:note`

`/bee:backlog`

Manage the seed backlog

Reviews seeds in *.bee/seeds/*, lets you promote ready seeds to specs, archive expired ones, or adjust their triggers. Run periodically to keep the backlog relevant.

When to use

- Weekly backlog review.
- After shipping a feature, to see which seeds are now unblocked.

Examples

```
/bee:backlog
```

Related

`/bee:seed`, `/bee:new-spec`

/bee:note

Zero-friction note capture

Saves a quick note to *.bee/notes/*. Unlike seeds, notes are not intended as future work — they are reference material: decisions, learnings, links, observations. Browsable in the Bee Hive dashboard.

Arguments

`note` — Free-text note body

When to use

- Any observation worth keeping.
- Decisions made informally that you want on record.

Examples

```
/bee:note "we decided to keep pagination at 25 items because ""it matches the old system — revisi
```

Related

`/bee:seed`, `/bee:thread`

/bee:thread

Cross-session knowledge persistence

Maintains named threads of related knowledge that span multiple sessions and specs. Threads are long-running conversations with Bee about a topic — the data model, the auth story, the deploy pipeline — whose state survives past any single session.

When to use

- Topics that come up repeatedly across sessions and you want Bee to keep context for.
- Architecture discussions that span weeks.

Examples

```
/bee:thread
```

Related

`/bee:note`, `/bee:memory`

`/bee:workspace`

Parallel worktrees for independent features

Creates a new git worktree so you can work on a second feature in isolation from the first. Each worktree has its own *.bee/* state, so Bee keeps the specs separate. Useful when two features are being built in parallel by the same person.

Arguments

`new` — Optional 'new' to create a new workspace

When to use

- Two features in flight where one shouldn't slow down the other.
- Hotfix work that must ship from a clean branch while a feature is in progress.

Examples

```
/bee:workspace new hotfix-login
```

```
/bee:workspace
```

Related

`/bee:init`

Chapter 24

Visualization: The Bee Hive Dashboard

Everything Bee writes is in *.bee/* as markdown or JSON. That makes it inspectable from a terminal, but reading a long review report in less is a chore. The Bee Hive dashboard is a local web UI that reads the same files and renders them nicely. There is one command.

`/bee:hive`

Start or stop the local Bee Hive dashboard

Starts a zero-dependency Node.js HTTP server at *http://localhost:3333* that serves an IDE-style view of the *.bee/* directory: three-column layout, file tree, tabbed main area, activity feed, roadmap timeline, split pane for side-by-side comparisons. The server binds only to *127.0.0.1* (no remote exposure) and shuts down automatically when the Claude Code session exits.

Arguments

`stop` — Pass 'stop' to shut the server down explicitly

When to use

- Reading long review or audit reports — reading them in a browser is much nicer than in a terminal.
- Browsing seeds, notes, or archived specs.
- Showing a colleague the state of a feature.
- Editing config values via inline toggles.

Examples

```
/bee:hive
```

```
/bee:hive stop
```

Related

`/bee:progress`, `/bee:health`

Chapter 25

Extensibility Commands

Bee ships with a large default set of agents and skills, but every project eventually has needs that the defaults don't cover. These two commands let you extend Bee without modifying the plugin itself — your extensions live in your project's *.bee/* directory and travel with the repo.

`/bee:create-agent`

Create a project-local agent

Scaffolds a new agent inside *.bee/agents/* with the correct frontmatter and a starting system prompt. The agent becomes available to commands that name it. Use this for custom reviewers, custom auditors, or specialized implementers.

Arguments

`name` — The new agent's name

When to use

- You want a reviewer that focuses on your team's specific style rules.
- You need an auditor for a domain that Bee's defaults don't cover (e.g. a compliance regime specific to your industry).
- You're iterating on a specialized implementer for a niche stack.

Examples

```
/bee:create-agent hipaa-auditor
```

Related

`/bee:create-skill`

/bee:create-skill

Create a project-local skill

Scaffolds a new skill under *.bee/skills/* with the correct shape: SKILL.md, optional reference files, frontmatter matching Bee's skill format. The skill can then be attached to agents that should have access to its knowledge.

Arguments

`name` — The new skill's name

When to use

- Your team has conventions that aren't captured by Bee's default standards and you want them injected into every relevant agent.
- You use a library Bee doesn't have a skill for and you want to teach agents how to use it idiomatically.

Examples

```
/bee:create-skill internal-style-guide
```

Related

`/bee:create-agent`

Part V

The Agent Roster

Every specialist Bee calls on, and why

Chapter 26

How to Read This Part

Part V catalogs Bee's 39 agents grouped by role. For each agent you get its name, its job, what it reads, what it writes, what the SubagentStop validator checks for, and which commands summon it. You will rarely call an agent directly — agents are invoked by commands — but knowing who does what helps you understand what a command is actually doing and gives you the vocabulary to debug problems when an agent misbehaves.

The general agent contract

Every Bee agent is constrained by the same high-level contract:

- Has one job. Agents that try to do two things produce worse output than agents that do one thing well.
- Has a specified output format, enforced by a SubagentStop validator. If the output doesn't match the contract, the agent re-runs.
- Cannot commit. Cannot run destructive tools without justification. Cannot modify *.bee/STATE.md* directly (only conductors can).
- Runs with tools restricted to what it needs. An auditor cannot Write or Edit; a planner cannot run Bash; a researcher cannot modify tests.

Chapter 27

Implementation Agents

The workers who actually write code.

implementer

The full-stack TDD worker. Handles one task at a time, running the full red-green-refactor cycle. Uses scoped testing — only runs tests relevant to its task's files.

Model

Opus in quality and premium modes, Sonnet in economy mode

Reads

- TASKS.md entry for its task
- Relevant stack skill
- Project CONTEXT.md
- User preferences (.bee/user.md)

Writes

- Production code
- Tests for its task
- Task checkbox update
- Research notes in TASKS.md

Validator checks

Verifies a red-to-green test transition, that the task checkbox is updated, that the output format matches the contract, and that no files outside the task's scope were modified.

Summoned by

`/bee:execute-phase`

quick-implementer

TDD-enforced implementer for quick tasks. Same discipline as the full implementer but scoped to a single quick task rather than a phase.

Model

Same as implementer

Reads

- Quick task description
- Project CONTEXT.md

Writes

- Production code + test

- Quick task record in .bee/quick/

Validator checks

Same validator as implementer.

Summoned by

`/bee:quick`

fixer

Applies targeted fixes from review or audit reports. Reads findings, investigates root cause, and applies minimal changes that address the finding without introducing new issues.

Model

Opus in quality and premium modes

Reads

- REVIEW.md or AUDIT-REPORT.md
- The code being fixed

Writes

- Fix report with finding ID, status, files changed, and tests updated
- Actual code and test changes

Validator checks

Each fix must reference a finding by ID, include test evidence, and produce a status of FIXED, PARTIAL, or UNFIXABLE with reasoning.

Summoned by

`/bee:fix-implementation`, `/bee:review (loop mode)`

Chapter 28

Planning and Research Agents

The thinkers who turn a spec into a plan and extract knowledge from your codebase.

researcher

Scans the codebase for patterns, conventions, and reusable code. Answers questions like 'how do we handle errors in this project?' or 'what's the idiomatic way to add a new API endpoint?'. Uses Context7 when available for live library documentation.

Model

Sonnet in quality, Opus in premium

Reads

- Codebase (read-only)
- Context7 docs
- CONTEXT.md

Writes

- Research notes in TASKS.md
- Pattern references with file:line
- Reusable code suggestions

Validator checks

Output must be structured research findings or TASKS.md updates; empty output is only valid with explicit 'scan complete, no relevant findings' signal.

Summoned by

`/bee:plan-phase`, `/bee:quick`

phase-planner

Decomposes a phase into concrete tasks with wave assignment. Reads the spec, reads the research from researcher, and produces a partial-order task list where tasks in the same wave can run in parallel without conflict.

Model

Sonnet in economy, Opus in premium

Reads

- spec.md and phases.md
- Research from researcher
- Project CONTEXT.md

Writes

- TASKS.md with per-phase task list, wave assignment, acceptance criteria, and test scenarios

Validator checks

Task list must be non-empty, must include wave numbers, and must have acceptance criteria for each task.

Summoned by

`/bee:plan-phase`, `/bee:plan-all`

spec-writer

Writes spec.md, phases.md, and ROADMAP.md from the Q&A; output of spec-shaper. Enforces the no-implementation rule: the spec must describe behaviour, not code.

Model

Sonnet in quality, Opus in premium

Reads

- Q&A; transcript from spec-shaper
- User profile

Writes

- spec.md
- phases.md
- ROADMAP.md
- requirements.md

Validator checks

spec.md must have all standard sections; must not contain file paths, class names, or implementation specifics.

Summoned by

`/bee:new-spec`

spec-shaper

Runs the adaptive discovery conversation. Asks questions based on what it learns, reads the codebase to verify claims, and signals when discovery is complete.

Model

Opus in quality and premium, Sonnet in economy

Reads

- User's free-text description
- Codebase (read-only)
- Existing spec (in amend mode)

Writes

- Q&A; transcript as structured requirements

Validator checks

Must conclude with either a 'ready for spec-writer' signal or a list of remaining open questions.

Summoned by

`/bee:new-spec`

assumptions-analyzer

Validates pre-planning assumptions — things the spec or plan implicitly assumes about the codebase or environment. Each assumption gets a confidence, impact, risk, and mitigation.

Model

Sonnet

Reads

- Spec, plan, codebase

Writes

- Structured assumption list

Validator checks

Every assumption must have confidence (high/medium/low), impact, risk, mitigation.

Summoned by

`/bee:plan-phase (research mode)`

dependency-auditor

Checks phase-relevant dependencies for age, security advisories, and version drift. Reports a phase dependency health summary.

Model

Sonnet

Reads

- package.json, composer.json, and their lockfiles

Writes

- Phase dependency health report

Validator checks

Must produce a structured report with phase-relevant dependencies listed separately.

Summoned by

`/bee:plan-phase (when context7 is enabled)`

Chapter 29

Review Agents

The specialists who check the code before it gets committed.

bug-detector

Scans code for logic errors, security vulnerabilities, error handling gaps, and off-by-one-class mistakes. Reports findings with file:line, evidence, impact, and test gap.

Model

Opus in quality and premium

Reads

- The phase's code changes
- Related tests

Writes

- Findings in REVIEW.md

Validator checks

Only HIGH-confidence findings; every finding must include file:line, Evidence, Impact, and Test Gap fields.

Summoned by

`/bee:review`, `/bee:swarm-review`

pattern-reviewer

Compares code against established project patterns found in CONTEXT.md. Flags deviations with 'existing pattern vs your pattern' comparisons and suggests alignment.

Model

Opus in quality and premium

Reads

- The phase's changes
- CONTEXT.md

Writes

- Findings in REVIEW.md

Validator checks

Each finding must show the existing-pattern code and the current-code side by side.

Summoned by

`/bee:review`, `/bee:swarm-review`

plan-compliance-reviewer

Checks whether the code actually implements what the plan said, task by task. Flags tasks marked done without matching code, tasks whose code deviates from their acceptance criteria, and spec requirements that aren't yet implemented.

Model

Opus in quality and premium

Reads

- spec.md
- TASKS.md
- The phase's code changes

Writes

- Coverage matrix in REVIEW.md

Validator checks

Produces either a coverage matrix or a structured list of spec gaps.

Summoned by

`/bee:review`, `/bee:review-implementation`

stack-reviewer

Runs framework-specific best-practice checks using the active stack skill. Catches anti-patterns that are specific to the framework — e.g. missing Suspense boundaries in Next.js, inappropriate N+1 patterns in Laravel, wrong Pinia store scopes in Vue.

Model

Opus in quality and premium

Reads

- The phase's code changes
- The active stack skill

Writes

- Stack-specific findings in REVIEW.md

Validator checks

Findings must reference a specific best practice and cite the relevant stack skill section.

Summoned by

`/bee:review`, `/bee:swarm-review`

finding-validator

The filter for review findings. Reads each raw finding, re-reads the actual code, and classifies it as REAL_BUG, STYLISTIC, or FALSE_POSITIVE. Only real bugs survive to REVIEW.md.

Model

Opus in quality and premium

Reads

- Raw findings from other review agents
- The actual code

Writes

- Filtered findings in REVIEW.md

Validator checks

Every classified finding has a justification; every filtered-out finding has a reason.

Summoned by

`/bee:review`

plan-reviewer

Reviews a planned phase against the spec. Looks for missing tasks, incorrect wave assignments, unrealistic dependencies, and gaps in acceptance criteria. Can also re-run to re-review after auto-fixes.

Model

Sonnet in quality, Opus in premium

Reads

- spec.md, phases.md, TASKS.md, PLAN.md

Writes

- PLAN-REVIEW.md

Validator checks

Output must be Approved or Issues Found with a specific findings list.

Summoned by

`/bee:plan-phase`, `/bee:plan-review`

spec-reviewer

Reviews a newly-written spec for completeness, consistency, clarity, and YAGNI. Flags missing success criteria, contradictions between requirements, ambiguous phrasing, and scope creep.

Model

Sonnet in quality, Opus in premium

Reads

- spec.md and phases.md

Writes

- Spec review notes for the spec loop

Validator checks

Output must be Approved or Issues Found.

Summoned by

`/bee:new-spec`

swarm-consolidator

Merges results from parallel segmented swarm reviews. Deduplicates findings across segments, re-ranks by severity, and produces a single consolidated report.

Model

Opus in quality and premium

Reads

- Segment review outputs

Writes

- Consolidated REVIEW.md

Validator checks

Must produce a single merged report; duplicates must be counted and collapsed.

Summoned by

`/bee:swarm-review`

Chapter 30

Audit Agents

The specialists used by */bee:audit* and */bee:review-implementation*. Each one owns a different dimension of code quality and has its own finding ID prefix.

security-auditor (`F-SEC`)

OWASP Top 10 patterns, auth bypass, injection (SQL, XSS, SSRF, template, command), secrets exposure, insecure deserialization, CORS/CSRF misconfigurations, weak crypto, missing rate limits. Scope includes both application and configuration files.

error-handling-auditor (`F-ERR`)

Missing error handling, crash vectors, silent failures, happy-path-only code, swallowed exceptions, unclosed resources. Focuses on resilience and the absence of code rather than its presence.

database-auditor (`F-DB`)

N+1 queries, missing indexes, unsafe migrations, missing transactions, incorrect constraints, data integrity risks, schema design anti-patterns, ORM misuse, connection lifecycle issues.

architecture-auditor (`F-ARCH`)

Layer separation violations, god files, code duplication, cyclic dependencies, boundary leaks, inconsistent error types across layers, and misuse of architectural patterns.

api-auditor (`F-API`)

Endpoint input validation, response schema consistency, auth on every route that needs it, CORS policy, rate limiting, error response shape, versioning strategy, backward compatibility.

frontend-auditor (`F-FE`)

Loading states, empty states, error states, accessibility (focus, labels, semantic HTML, keyboard flow, ARIA), responsive breakpoints, form handling, controlled/uncontrolled pitfalls, memory leaks in useEffect / watchers, performance budgets.

performance-auditor (`F-PERF`)

Backend bottlenecks (synchronous I/O in hot paths, unbatched requests, missing caches), frontend optimizations (bundle size, image formats, code splitting, hydration cost), async bottlenecks, resource lifecycle.

testing-auditor (F-TEST)

Coverage gaps by feature area, test quality (assertion strength, flaky patterns, implicit dependencies), stale tests that no longer match the code, test infrastructure health, CI test runtime drift.

audit-bug-detector (F-BUG)

End-to-end flow tracing across layers. Catches the cross-cutting bugs that per-phase reviews miss — data contract mismatches between phases, dependency chain breaks, inconsistent validation at layer boundaries.

audit-finding-validator (–)

Validates every raw audit finding against the real code before it enters the report. Classifies as CONFIRMED, FALSE POSITIVE, or NEEDS CONTEXT. False positives never reach AUDIT-REPORT.md.

audit-report-generator (–)

Merges validated findings from all auditors into AUDIT-REPORT.md (human-readable) and audit-findings.json (machine-readable). Handles severity calibration and cross-agent deduplication.

Chapter 31

Debug, Specialized, and Stack-Specific Agents

debug-investigator

Structured hypothesis-driven bug investigation. Generates ranked hypotheses, investigates each against the code, tracks evidence, and maintains a living INVESTIGATION.md.

Model

Opus in quality and premium

Reads

- Bug description
- Codebase (read-only)
- Test output / logs

Writes

- INVESTIGATION.md in .bee/debug/

Validator checks

Output must track hypotheses with status (open/ruled out/confirmed) and evidence references.

Summoned by

`/bee:debug`

integrity-auditor

Compares STATE.md against actual disk reality. Flags phase states that don't match the code, tasks marked done without evidence, and missing files that STATE references.

Model

Sonnet

Reads

- STATE.md
- The full .bee/ tree
- The working directory

Writes

- Integrity report in EOD or HEALTH output

Validator checks

Structured report of discrepancies or 'all clean' signal.

Summoned by

`/bee:eod`, `/bee:health`

testing-auditor (the health variant)

Audits the test suite for health metrics separate from the audit-phase version: coverage trends, flaky tests, slow tests, redundant tests, and CI test-time drift.

Model

Sonnet

Reads

- Test files
- Test runner config
- CI config

Writes

- Test health report for EOD and health reports

Validator checks

Same format as the audit version.

Summoned by

`/bee:eod`, `/bee:health`

test-auditor

Audits test quality: assertion strength, setup/teardown hygiene, mocking discipline, anti-patterns like test interdependence and hidden order dependencies.

Model

Sonnet

Reads

- Test files

Writes

- Test quality report

Validator checks

Structured report or 'all clean'.

Summoned by

`/bee:eod`

test-planner

Generates manual test scenarios for a phase. Reads the spec's acceptance criteria and produces step-by-step scenarios a human (or QA) can run.

Model

Sonnet in quality, Opus in premium

Reads

- spec.md
- TASKS.md
- CONTEXT.md

Writes

- Manual test scenarios for /bee:test

Validator checks

Scenarios must be numbered, must have expected results, must cover all acceptance criteria.

Summoned by

`/bee:test`

context-builder

Scans the codebase and extracts patterns, conventions, and reusable primitives into CONTEXT.md. Run at init and at /bee:refresh-context.

Model

Sonnet in quality, Opus in premium

Reads

- The full codebase

Writes

- CONTEXT.md

Validator checks

CONTEXT.md must cover structure, conventions, entry points, naming, and error-handling style.

Summoned by

`/bee:init`, `/bee:refresh-context`

discuss-partner

The codebase-grounded brainstorming conductor. Runs in two modes: scan (read the codebase for context) and write-notes (capture the discussion).

Model

Opus in quality and premium

Reads

- User's topic description
- Codebase (read-only)

Writes

- Discussion notes in .bee/discussions/

Validator checks

Must either write notes or signal scan complete.

Summoned by

`/bee:discuss`

ui-auditor

Six-pillar UI audit: layout, typography, colour + contrast, responsiveness, interaction states, accessibility. Generates findings per pillar with concrete remediation.

Model

Opus

Reads

- The UI code
- ui-spec.md if present

Writes

- UI audit report for /bee:ui-review

Validator checks

Each pillar must have its own findings section; missing sections fail validation.

Summoned by

`/bee:ui-review`

integration-checker

End-to-end flow tracing across layers. Picks a user story and walks it through UI, API, service layer, data layer, flagging contract mismatches and missing pieces.

Model

Opus

Reads

- Spec
- Codebase (read-only)

Writes

- Integration report

Validator checks

Must trace at least one end-to-end flow; must cite file:line at each layer crossing.

Summoned by

`/bee:audit`, `/bee:review-implementation`

Stack-specific variants

Some stacks ship variant agents tuned for their idioms. The *laravel-inertia-vue* stack, which is Bee's gold-standard reference implementation, ships three: its own *implementer*, *bug-detector*, and *pattern-reviewer*. When a Laravel+Vue project runs */bee:execute-phase* or */bee:review*, these stack-specific agents replace the generic ones automatically.

The same extension pattern is available for your own project-local agents — see Chapter 25 (*/bee:create-agent*).

Part VI

Skills, Stacks, and Libraries

The knowledge bases that make Bee aware of your stack

Chapter 32

Core and Standards

Every agent in Bee can load one or more skills. Skills are bundled knowledge attached to an agent's system prompt at runtime. Bee's skills fall into five layers, loaded conditionally:

- **Core** — the baseline rules every Bee agent obeys. Always loaded.
- **Standards** — quality rules (testing, frontend, backend, auth) loaded based on what the current task touches.
- **Stacks** — per-stack best practices, loaded based on the detected stack.
- **Libraries** — per-library patterns, loaded based on manifest detection.
- **Support** — audit rules, review rules, context7, and the Bee mastery guide, loaded by specific commands.

The core skill

The core skill defines the firm rules R1 through R9 that every agent obeys. R1 is the TDD iron law. R2 is disk-is-truth. R3 bans auto-commit. R4 establishes scoped testing. R5 requires evidence for task completion. R6 establishes the sub-agent contract. R7 enforces output format. R8 establishes the model tiering mapping. R9 establishes context7 behaviour.

Every agent loads the core skill. You cannot disable it. It is what makes a Bee agent a Bee agent.

Standards

Standards are always-on quality layers that add on top of the core rules. The current set:

Standard	Scope
testing	TDD patterns, async testing, flaky test prevention, fixture discipline
frontend	Component architecture, accessibility, responsive design, Core Web Vitals
backend	API design, error handling, input validation, layering, persistence
auth	Authentication and authorization patterns, session lifecycle, token safety
(others)	Realtime, CI/CD, Docker, monorepo, i18n loaded conditionally

When Bee runs */bee:review* on a frontend-heavy phase, the frontend and testing standards load automatically. When the phase is backend-heavy, backend and testing load instead. When auth is touched, auth also loads. You never have to configure this — it is driven by what files the phase actually modifies.

Chapter 33

Stack Skills

Bee ships with 10 stack skills. Each one is a long-form best-practice document for its framework, written specifically for agents that will be generating code in that framework. Stack skills load automatically based on */bee:init* stack detection.

Stack	Detection	Notes
laravel-inertia-vue	composer.json + vue	Gold standard, 3 stack-specific agents
laravel-inertia-react	composer.json + react	React 19, useForm, Inertia deep dive
nestjs	@nestjs/core	Custom decorators, Prisma/TypeORM, GraphQL, Swagger
vue	vue without Laravel	Composition API, Pinia, Vue Router 4, VeeValidate + Zod
nextjs	next	App Router, Server Actions, caching
react	react without next/expo	Hooks, Router v7, Suspense
angular	@angular/core	Signals, standalone, OnPush, inject
react-native-expo	expo + react-native	Expo Router, platform-specific patterns
kmp-compose	build.gradle.kts multiplatform	Compose Multiplatform, Voyager, Koin, Ktor
claude-code-plugin	manual	For building Bee itself

A stack skill is not just 'here's some syntax'. It is a document that encodes opinions: where files go, how to name things, which patterns are idiomatic and which are legacy, what the version-current way to do X is. When you run */bee:review* on a Next.js project, the stack-reviewer loads the nextjs skill and flags anti-patterns specific to that framework.

Chapter 34

Library Skills

Library skills are the same shape as stack skills but scoped to a single library rather than a whole framework. They load conditionally based on *package.json* or *composer.json* detection, so a project that uses Prisma gets the Prisma skill injected into relevant agents but a project that doesn't stays unencumbered.

Current library skills:

Library	Detection
prisma	@prisma/client in package.json
drizzle	drizzle-orm in package.json
tailwind-v4	tailwindcss ^4 in package.json
shadcn-ui	components.json or @/components/ui/
shadcn-vue	shadcn-vue in package.json
tanstack-query	@tanstack/react-query
zustand	zustand in package.json
trpc	@trpc/client or @trpc/server
stripe	stripe in package.json
supabase	@supabase/supabase-js
sentry	@sentry/* in package.json
storybook	@storybook/* in package.json
email	resend, sendgrid, or nodemailer detected
s3-storage	@aws-sdk/client-s3 or similar detected
nestjs-rabbitmq	@nestjs/microservices + amqplib
playwright	invoked by /bee:test-e2e

Chapter 35

How Skills Load

Skill loading is not a surprise. It follows a fixed procedure whenever an agent is spawned:

- The core skill is attached unconditionally.
- The standards that match the task's file pattern attach. If the task modifies **.vue* or **.tsx*, the frontend standard attaches. If it modifies **.php* or **.ts* under *src/api*, backend attaches. If it touches auth code, auth attaches.
- The active stack skill attaches, based on the path the task lives under in multi-stack projects.
- Library skills attach for every library that manifest detection matched.
- Command-specific skills (audit, review, context7) attach only when a command that uses them runs the agent.

The upshot is that a single *implementer* agent on a Next.js + Prisma + Tailwind v4 + shadcn project will load core, testing, frontend (and maybe backend), the nextjs stack skill, and the prisma, tailwind-v4, and shadcn-ui library skills automatically. The total skill context is non-trivial, but it means the agent has the knowledge it needs to produce idiomatic code without you having to prompt it.

Part VII

Workflows and Situations

End-to-end recipes for real-world work

Chapter 36

The Full Feature Workflow

This is the canonical workflow. You use it when you are building a substantial feature from scratch, when you want every piece of work to go through review, and when you have the time to run the pipeline step by step.

When to use it

- New features bigger than half a day's work.
- Features where the spec is non-trivial and needs careful shaping.
- Team projects where the process itself is the deliverable (not just the code).
- Any time you want the maximum amount of review coverage.

The steps

```
# Once per project
/bee:init

# Optional: brainstorm before spec
/bee:discuss

# Create and review the spec
/bee:new-spec
# Read spec.md, confirm, or amend with /bee:new-spec --amend

# For each phase 1..N:
/bee:plan-phase 1
# Inspect TASKS.md, re-review if needed with /bee:plan-review 1
/bee:execute-phase 1
/bee:review
# Inspect REVIEW.md; /bee:fix-implementation if needed
/bee:test
/bee:commit

# After all phases:
/bee:review-implementation
/bee:complete-spec
```

Why this order

Each step feeds the next and every step has a gate. The spec gate catches misunderstood requirements before they become planned work. The plan gate catches bad decomposition before it becomes wrong code. The review gate catches bad code before it becomes bad commits. The implementation review catches cross-phase drift before it becomes a feature release. Skipping any gate saves time now at the cost of multiplier risk later.

Common variations

- Start with */bee:discuss* if you are not sure about the shape of the feature yet.

- Use */bee:plan-all* if you trust your spec and want to batch planning.
- Use */bee:swarm-review* instead of */bee:review* on very large phases.
- Add */bee:ui-spec* and */bee:ui-review* around frontend-heavy phases.
- Add */bee:test-e2e* after */bee:test* for user-facing flows.

Chapter 37

The Autonomous Ship Workflow

For when you trust the spec and want to walk away. */bee:ship* is the heaviest-lifting command in Bee. It is designed to run for an extended period without human input and leave you with a pile of reviewed, tested, uncommitted changes.

When to use it

- The spec is stable and well-specified.
- The phases are independently deliverable and well-ordered.
- You want Bee to run unattended while you do something else.
- You are comfortable reviewing a big batch of changes at the end instead of incremental ones.

The steps

```
/bee:init                  # once per project
/bee:new-spec              # adaptive discovery + review
/bee:plan-all              # plan all phases + cross-plan review
/bee:ship                  # autonomous execution + per-phase review
# walk away, come back later
/bee:review-implementation # final cross-phase compliance check
/bee:commit                # human-approved batch commit
/bee:complete-spec         # close and archive
```

What to watch for

- Every decision */bee:ship* makes is logged in STATE.md. When you return, read the decision log — it is the paper trail for everything that happened.
- Ship will not auto-commit. You always review the diff before anything lands in git.
- If the spec has ambiguity the spec-reviewer and plan-reviewer missed, ship may make judgment calls you disagree with. This is why the decision log exists.
- If a phase's review loop hits its maximum iterations without converging, ship continues optimistically — it prefers progress with a note over blocking.

Chapter 38

The Quick-Task Workflow

Sometimes you just need to fix a button. You don't want to spec it, you don't want to plan it, you want it done. */bee:quick* is the escape valve for that case, and it has three modes depending on how much process you want.

Default mode — TDD with agents

```
/bee:quick "fix the login button alignment on mobile"
```

Bee launches a researcher to scan for relevant patterns, then a quick-implementer that does the red-green-refactor cycle on the task. This is the right choice for bug fixes and small features that deserve a test.

Fast mode — direct edits

```
/bee:quick --fast "update the footer copyright to 2026"
```

Skips agents entirely, edits directly. Use only for truly trivial changes you would not have thought to test even without Bee. This is an escape hatch; abuse it and you lose Bee's main value.

Review mode — execute + 4-agent review

```
/bee:quick --review "refactor the auth middleware"
```

Runs TDD execution, then the full 4-agent review before commit. Use this when a 'quick' task touches sensitive code or deserves an extra set of eyes despite not being big enough for a full spec.

Amending a quick task

```
/bee:quick --amend 3
```

Quick tasks are stored in *.bee/quick/* by number. *--amend* re-opens task N and lets you modify its plan or description. Useful for iterating on a fix.

Chapter 39

The Audit and Takeover Workflow

You just inherited a codebase you didn't write. It probably has bugs. It definitely has questionable decisions. You want a structured view of what is wrong, and you want to fix the important things without getting lost in the merely ugly ones.

The steps

```
/bee:init                    # initialize Bee on the project
/bee:audit                   # 10 auditors scan in parallel
/bee:audit-to-spec --critical  # convert CRITICAL findings to specs

# For each generated spec:
/bee:new-spec --from-discussion .bee/audit-specs/critical-NN.md
/bee:plan-phase 1
/bee:execute-phase 1
/bee:review
/bee:commit

# When criticals are done, do HIGH and MEDIUM:
/bee:audit-to-spec --high
# (repeat the pipeline)
```

Tips for takeovers

- Start by reading AUDIT-REPORT.md top to bottom. Don't jump straight into fixing — understand the landscape first.
- Use */bee:audit --only security,database* if the project is huge and you want to start narrow.
- If an audit finding looks wrong, add it to *.bee/false-positives.md*. The validator will respect it on future runs.
- Don't fix MEDIUM and LOW findings until you understand them — they may be intentional.
- Run */bee:health* periodically during the takeover to track progress and detect regressions.

Chapter 40

The Debug and Recovery Workflow

Something is broken and you can't figure out why. You've already tried the obvious things. Time for a structured investigation.

Step 1 — Forensics, if something went wrong in a workflow

```
/bee:forensics
```

If the bug is 'this phase shipped something I didn't expect', forensics is where you start. It produces a read-only forensic report with a 4-factor severity rubric that tells you which phases to investigate more deeply.

Step 2 — Debug session

```
/bee:debug
```

Describe the bug to the debug-investigator agent. It will generate ranked hypotheses, investigate each one, and track evidence in *.bee/debug/NNNN-slug/INVESTIGATION.md*. You can pause the session and resume later with */bee:debug --resume*.

Step 3 — Health baseline

```
/bee:health
```

Before fixing the bug, capture a health baseline. After the fix, run */bee:health* again — the trend arrows will show whether your fix improved the underlying health or just papered over a symptom.

Step 4 — Fix via the normal pipeline

Once the bug is isolated, treat the fix like any other piece of work. Small fix: */bee:quick --review*. Medium fix: a new spec or an inserted phase in an existing spec via */bee:insert-phase*. The investigation in *.bee/debug/* becomes the reference for the fix's spec.

Chapter 41

The Inherited Project Situation

Closely related to the audit workflow, but with more human elements. You inherited a project, you don't know its history, and you need to start shipping features without breaking things.

First day

- */bee:init* — stack detection, context extraction.
- Read *.bee/CONTEXT.md* end to end. This is your orientation.
- */bee:health* — capture a baseline.
- */bee:audit --only security,database,architecture* — hit the load-bearing dimensions first.

First week

- */bee:audit-to-spec --critical* and start addressing the critical findings with small, boring specs.
- Use */bee:discuss* liberally to sanity-check assumptions with the codebase.
- Run */bee:eod* every day. The velocity and integrity signals will tell you whether you're actually making progress or just churning.

First month

- You should be past the audit backlog for criticals and making progress on highs.
- Your CONTEXT.md will need a refresh — run */bee:refresh-context*.
- Start shipping planned features with full specs. The takeover is over; you're now the maintainer.

Chapter 42

The Multi-Stack Monorepo

One repository, two or three apps, each on a different stack. Bee handles this natively through the multi-stack config, but there are a few things to watch for.

Init detection

Run */bee:init* at the repo root. Bee scans first-level subdirectories for manifests and offers a stack-path list. Confirm or adjust each entry. Your *config.json* ends up looking like:

```
{
  "stacks": [
    { "name": "laravel-inertia-vue", "path": "." },
    { "name": "nestjs", "path": "api" },
    { "name": "vue", "path": "admin" }
  ],
  "implementation_mode": "quality"
}
```

Spec scope in monorepos

A spec can touch multiple stacks, or it can be scoped to one. When the spec is multi-stack, the phase-planner will produce tasks for each stack, and execute-phase will spawn stack-aware implementers for each. Stack-specific agents load automatically per task.

Tips

- Keep specs scoped when you can — 'add auth to the admin app' is easier to manage than 'add auth everywhere'.
- When a spec legitimately spans stacks, be explicit in the phase breakdown about which stack each phase lives in.
- Use */bee:workspace* to spin up a parallel worktree if one app is being refactored heavily and you don't want it to block work on the others.

Chapter 43

The Crash and Resume Story

Bee is designed to survive crashes, power loss, and accidental session closes. The mechanism is the disk-is-truth principle: as long as *.bee/* is intact, you can recover. This chapter walks through what happens in practice.

What gets saved, when

Every conductor writes state before it considers its work done. Before */bee:execute-phase* finishes a wave, it updates TASKS.md. Before */bee:review* finishes, it writes REVIEW.md. Before */bee:commit* commits, it updates STATE.md. If you lose the session between waves, you lose at most the work of the current wave — never the work of previous waves.

How to recover

```
# Open a new Claude Code session and run:
/bee:resume
```

Bee reads STATE.md and prints a 'resuming at X' message. If you were mid-execute, it will offer to continue the current phase from the last wave boundary. If you were mid-review, it will offer to re-run the review. If you were mid-debug, */bee:debug --resume SESSION_ID* will continue the investigation.

When things really break

If *.bee/STATE.md* itself is corrupted — truncated, partially written, or has conflicting status rows — run */bee:forensics*. It is read-only and will diagnose the inconsistency without making things worse. From the forensic report you can manually repair STATE.md, or in bad cases start over with the spec intact and re-plan the current phase.

Chapter 44

Situational Recipes

Short recipes for problems you will actually face. Each one is a pattern, not a command — combine Bee's primitives in this order for this situation.

I want to add one small thing to an existing phase

```
/bee:insert-phase 2.1
/bee:plan-phase 2.1
/bee:execute-phase 2.1
/bee:review
/bee:commit
```

Inserting a decimal-numbered phase keeps the main numbering stable and gives the small thing its own review gate.

I want to capture an idea without stopping current work

```
/bee:seed "when we migrate to Postgres, switch to JSONB for ""the settings blob"
```

Seeds don't interrupt flow and they don't get lost.

I fixed a bug manually and want a regression test

```
/bee:quick --review "add regression test for the saved-item duplicate bug"
```

Quick mode with review gives you TDD + the 4-agent review without the spec overhead.

I want to audit only one dimension

```
/bee:audit --only security
```

Narrower scope runs faster and produces a focused report.

I want to pause and come back in a week

```
/bee:pause
/bee:eod   # optional but recommended
```

Then when you come back:

```
/bee:resume
/bee:health
```

The health diff will show you anything that changed while you were away.

I'm blocked on a decision and want to think with the AI

```
/bee:discuss "should we use tRPC or keep REST"
```

Discuss is lower-pressure than new-spec and leaves notes you can turn into a spec later.

I want to show my teammate what Bee did this week

```
/bee:hive
```

Open localhost:3333, point at the roadmap timeline and the relevant spec directory.

A review agent keeps flagging the same false positive

Add it to *.bee/false-positives.md*. Every future review run will respect the dismissal. Include a short reason so future-you (or a teammate) can understand the call.

I want Bee to follow a personal convention everywhere

```
/bee:profile
```

Edit *.bee/user.md*. The SubagentStart hook will inject your preferences into every subsequent sub-agent.

Part VIII

Advanced Topics

Configuration, customization, troubleshooting, and anti-patterns

Chapter 45

Configuration Deep Dive

Everything in *.bee/config.json* is explained here. This file is edited indirectly by */bee:init* and the Bee Hive dashboard, but you can also edit it by hand when you know what you want.

The full schema

```
{
  "stacks": [
    {
      "name": "laravel-inertia-vue",
      "path": ".",
      "linter": "pint",
      "testRunner": "pest"
    }
  ],
  "implementation_mode": "quality",
  "ci": "github-actions",
  "context7": true,
  "research_policy": "recommended",
  "review": {
    "against_spec": true,
    "against_standards": true,
    "dead_code": true,
    "loop": false,
    "max_loop_iterations": 3
  },
  "phases": {
    "require_review_before_next": true
  },
  "ship": {
    "max_review_iterations": 3,
    "final_review": true
  },
  "quick": {
    "review": false,
    "fast": false
  },
  "metrics": {
    "enabled": true
  },
  "autonomous": {
    "discuss": true,
    "auto_approve_confidence": "none"
  },
  "adaptive": {
    "learning": true,
    "escalation": true
  }
}
```

The important knobs

- **implementation_mode** — economy, quality (default), or premium. Trade money for reasoning quality.
- **review.loop** — when true, */bee:review* automatically applies fixes and re-reviews until clean or the max iterations limit is hit. Great for autonomy, noisy during interactive sessions.
- **phases.require_review_before_next** — when true, you cannot execute phase N+1 until phase N has been reviewed. Default true; turn off only if you're sure.
- **ship.max_review_iterations** — how many review-fix rounds */bee:ship* will do per phase before giving up and continuing optimistically.
- **adaptive.learning** — when true, implementer writes learnings into LEARNINGS.md that future implementers read as context. Useful; turn off if it's too noisy.
- **research_policy** — 'required', 'recommended', or 'skip'. Controls whether */bee:plan-phase* runs the researcher agent.

Chapter 46

Hooks and Validators

Bee's hooks are the immune system — the things that happen automatically regardless of which command you're running. They are how Bee enforces its invariants without making you type anything.

The 8 lifecycle hooks

Hook	When	What Bee does
SessionStart	Session begins	Loads STATE.md, config, configures statusline
PostToolUse	After Write/Edit	Auto-lints the modified file
PreToolUse	Before Bash	Pre-commit validation gate
PreCompact	Before /compact	Snapshots state for recovery
SubagentStart	Agent spawns	Injects user preferences
SubagentStop	Agent finishes	Runs 27 role-specific validators
Stop	Session ends	Warns about unreviewed phases
SessionEnd	Session terminates	Emits session metrics

The 27 validators

The *SubagentStop* hook runs validators against an agent's output. Each validator is matched by the agent's name (via a regex on the subagent prompt, e.g. *implementer$*). The validator checks the output against the agent's contract. If the output is invalid, the agent is re-spawned with the validator's feedback until it produces valid output.

What validators catch, by category:

- **Implementer validators** — verify red-then-green test transition, verify task checkbox update, verify the task's files were actually modified, verify the output format matches the contract.
- **Review validators** — verify finding format (file:line, evidence, impact, test gap), verify confidence markings, verify no duplicates.
- **Audit validators** — verify finding ID prefix (F-SEC-NNN, F-DB-NNN, etc.), verify severity calibration, verify evidence citations.
- **Plan validators** — verify wave assignment, verify task non-emptiness, verify acceptance criteria.
- **Spec validators** — verify all required sections present, verify no implementation details leaked into the spec.

Validators are why Bee's agents behave consistently across runs. They are the reason you can trust the output format enough to build automation on top of it.

Chapter 47

Customizing Bee

Bee is extensible at four levels: configuration, personal profile, project-local agents, and project-local skills. Each level gets you more power at the cost of more responsibility.

Level 1 — config.json

The cheapest customization. Toggle options, change modes, tune review iterations. Edit through Bee Hive or by hand.

Level 2 — .bee/user.md

Personal preferences injected into every sub-agent via the SubagentStart hook. This is where you put 'always use arrow functions', 'prefer async/await over .then()', 'comments in English, never Romanian'. Personal to your Bee usage; shared only if you commit it.

Level 3 — Project-local agents

Created with */bee:create-agent* and stored in *.bee/agents/*. Travel with the repo. Use this when the default agent set doesn't cover a niche you care about — a compliance-specific auditor, a domain-specific bug detector, a specialized implementer for a niche framework.

Level 4 — Project-local skills

Created with */bee:create-skill* and stored in *.bee/skills/*. Use this when you want to teach Bee about a library or convention Bee doesn't know. The skill gets attached to whichever agents you reference it from.

Chapter 48

The Bee Hive Dashboard

The dashboard is a local web UI for browsing the *.bee/* directory. It is a read-mostly tool for reading reports in a nicer environment than a terminal, but it also supports a few write actions.

Layout

- **Left sidebar** — collapsible file tree organized by section: phases, notes, seeds, quick tasks, discussions, forensics, debug sessions, archived specs. Each section has a count badge and, where applicable, a status badge.
- **Main area** — tabbed markdown viewer. Click any file to open it in a tab with markdown rendering (headings, code blocks, tables, links).
- **Right sidebar** — live activity feed of agent events, updated every 5 seconds.

Useful views

- **Phase detail** — click a phase in the sidebar to open the rich phase view: workflow chain (plan → review → execute → test → commit), goal, deliverables, success criteria, requirements, and dependencies all in one place.
- **Roadmap timeline** — vertical timeline of all phases with clickable cards that jump to phase detail.
- **Split pane** — pop any tab into a second column with the \ key to compare two files side by side.

Endpoints

The server exposes three endpoints useful for extension or debugging:

Endpoint	Method	Purpose
/api/snapshot	GET	Full workflow state aggregator
/api/config	POST/PUT	Merge partial config changes
/api/file	GET	Read a .bee/ file with safety guards

The server binds to 127.0.0.1 only and shuts down when the owning Claude Code session exits, so there is no risk of leaving it exposed.

Chapter 49

Troubleshooting

'execute-phase' fails with 'phase not planned'

You skipped */bee:plan-phase N*. Run it first. If you think you already planned the phase, check STATE.md — the phase's status must be PLAN_REVIEWED (or PLANNED if you have review-before-next disabled) before execution is allowed.

'execute-phase' keeps failing the same task

Adaptive error recovery is retrying with different approaches. After the retries are exhausted, Bee will surface the failure with a summary of what was tried. Look at LEARNINGS.md — the details of each attempt are there. Common causes: a test that depends on something not yet implemented in this phase, a misnamed file path, a flaky assertion.

Review flagging things that are not actually bugs

The finding-validator is supposed to filter these, but it is not perfect. Add the finding to *.bee/false-positives.md* with a reason. Future reviews will ignore it.

Spec discovery feels like it's going in circles

The spec-shaper is adaptive — it keeps asking until it's confident. If you're going in circles, you probably have a contradiction in your answers. Slow down, re-read the last few questions, and give more precise answers.

/bee:ship made a decision I disagree with

Every ship decision is logged in STATE.md under 'decisions'. Open it, find the decision, and either amend the spec to forbid the choice next time or fix the code directly and re-run */bee:review*. Ship optimism is a tradeoff for autonomy; if you want tighter control, use */bee:autonomous* or the manual pipeline.

The dashboard won't start

/bee:hive runs a Node.js HTTP server on port 3333 (127.0.0.1). If something else is on 3333, the server will fail. Check *lsof -i :3333* to see what. The server also exits automatically when the Claude Code session that started it exits; if you can't stop a stale server, it probably means the owner session is still alive.

I accidentally ran /bee:init on an existing project

Safe. Re-init only updates config. STATE, specs, and user preferences are never touched. If you see a migration summary in the output, that means Bee noticed an old config format and migrated it in place.

Chapter 50

Anti-patterns

The Bee-specific foot-guns. Most of these are things that technically work but produce results you will regret.

Skipping the spec because 'it's obvious'

Whenever you catch yourself saying 'this is obvious, I'll just run quick', stop and ask whether the spec would really have taken more than 5 minutes. If the task is truly trivial, */bee:quick --fast* is fine. If there is any behaviour worth capturing, spec it — your future self will thank you when you need to understand why the code looks the way it does six months later.

Ignoring review findings because 'they're mostly noise'

If the review is mostly noise, fix the noise: add to *.bee/false-positives.md*, tune the stack skill, or file issue with Bee. If you ignore findings, you are ignoring the thing Bee is best at. Eventually a real bug slips through and you lose trust in the whole system.

Committing without */bee:commit*

The commit command isn't just a convenience wrapper. It verifies the diff summary, enforces the commit message convention, and marks the phase committed in STATE.md. Going around it leaves STATE.md inconsistent with git and can confuse the next command.

Hand-editing TASKS.md without re-reviewing

You can hand-edit TASKS.md — sometimes you need to. But if you do, run */bee:plan-review N* afterwards so the reviewer sees the edit. Silent edits lead to phases whose plan and review are out of sync.

Using */bee:quick --fast* for anything meaningful

--fast bypasses TDD and agents. It is for truly trivial edits. Using it for 'oh it's just a small bug fix' is how untested code sneaks in. If you would be uncomfortable shipping the change without tests, don't use *--fast*.

Running */bee:ship* on an unreviewed plan

/bee:ship assumes the plan is good. If the plan was not reviewed, ship will execute bad plans for hours. Always run */bee:plan-all* (which reviews every plan) before ship.

Mixing Bee work with untracked hand-written work

If you are editing files outside Bee while Bee is mid-phase, the SubagentStop validators may fail in surprising ways because the file state changes out from under the implementer. If you need to do hand edits, */bee:pause* first, do the edits, then */bee:resume*.

Part IX

Appendices

Indexes, glossary, and quick lookups

Chapter 51

Appendix A: Alphabetical Command Index

All 50 commands, alphabetized, with a one-line description. Use this when you know the command name and want a refresher.

Command	Description
/bee:add-phase	Append a new phase to the current spec
/bee:archive-spec	Quick archive without the full completion ceremony
/bee:audit	Comprehensive multi-agent code audit
/bee:audit-spec	Spec traceability matrix
/bee:audit-to-spec	Convert audit findings into actionable specs
/bee:autonomous	Plan + execute + review per phase autonomously
/bee:backlog	Manage the seed backlog
/bee:commit	Show diff, propose message, commit with human approval
/bee:compact	Smart context compaction preserving bee state
/bee:complete-spec	Full spec completion ceremony
/bee:create-agent	Create a project-local agent
/bee:create-skill	Create a project-local skill
/bee:debug	Systematic bug investigation with hypothesis tracking
/bee:discuss	Codebase-grounded brainstorming before a spec
/bee:do	Natural-language intent router
/bee:eod	End-of-day integrity check
/bee:execute-phase	Execute a planned phase with wave-based parallel TDD
/bee:fix-implementation	Apply fixes from a review report
/bee:forensics	Read-only workflow forensics
/bee:health	13-check project health diagnostics
/bee:help	Full command reference
/bee:hive	Start or stop the Bee Hive dashboard
/bee:init	Initialize Bee for a project
/bee:insert-phase	Insert a decimal-numbered phase mid-spec
/bee:memory	View accumulated agent memories
/bee:new-spec	Create or amend a feature spec
/bee:next	Suggest the next command
/bee:note	Zero-friction note capture
/bee:pause	Save work-in-progress for later

Command	Description
/bee:plan-all	Plan all remaining phases autonomously
/bee:plan-phase	Plan a single phase
/bee:plan-review	Standalone plan review
/bee:profile	View or edit user profile and preferences
/bee:progress	Show current project state
/bee:quick	Fast-track task without a full spec
/bee:refresh-context	Rebuild CONTEXT.md from scratch
/bee:resume	Restore full context after a break
/bee:review	Multi-agent parallel code review
/bee:review-implementation	Cross-phase spec compliance review
/bee:seed	Capture an idea with trigger conditions
/bee:ship	Execute all plan-reviewed phases fully autonomously
/bee:swarm-review	Deep segmented parallel review
/bee:test	Generate manual test scenarios
/bee:test-e2e	Playwright E2E tests with POM
/bee:test-gen	Requirement-driven test generation
/bee:thread	Cross-session knowledge persistence
/bee:ui-review	Six-pillar UI quality audit
/bee:ui-spec	UI design contract for a frontend phase
/bee:update	Update Bee statusline and cleanup
/bee:workspace	Parallel worktrees for independent features

Chapter 52

Appendix B: Agent Index

Agent	Role
implementer	Full-stack TDD implementation
quick-implementer	TDD-enforced implementer for quick tasks
fixer	Applies targeted fixes from review/audit reports
researcher	Codebase pattern research + Context7 docs
phase-planner	Decomposes phases into tasks and waves
spec-writer	Writes spec, phases, roadmap from Q&A
spec-shaper	Adaptive discovery Q&A conductor
spec-reviewer	Validates spec completeness and consistency
plan-reviewer	Reviews planned phases against spec
discuss-partner	Codebase-grounded brainstorming conductor
context-builder	Extracts patterns into CONTEXT.md
assumptions-analyzer	Validates pre-planning assumptions
dependency-auditor	Phase dependency health report
bug-detector	Logic errors and security issues
pattern-reviewer	Deviations from project conventions
plan-compliance-reviewer	Code-vs-plan compliance check
stack-reviewer	Framework best-practice violations
finding-validator	Filters review findings for accuracy
swarm-consolidator	Merges segmented swarm review outputs
security-auditor	OWASP, injection, secrets, auth issues
error-handling-auditor	Missing error handling, silent failures
database-auditor	N+1, indexes, migrations, transactions
architecture-auditor	Layer separation, god files, duplication
api-auditor	Endpoint validation, auth, CORS, responses
frontend-auditor	UI states, a11y, forms, performance
performance-auditor	Backend and frontend bottlenecks
testing-auditor	Coverage, test quality, stale tests
audit-bug-detector	Cross-layer end-to-end flow bugs
audit-finding-validator	Validates audit findings against code
audit-report-generator	Merges findings into AUDIT-REPORT.md
debug-investigator	Hypothesis-driven bug investigation
integrity-auditor	STATE vs disk consistency checks
test-auditor	Test quality audit

Agent	Role
test-planner	Generates manual test scenarios
ui-auditor	Six-pillar UI quality audit
integration-checker	End-to-end flow tracing
laravel-inertia-vue/implementer	Laravel+Vue TDD implementer
laravel-inertia-vue/bug-detector	Laravel+Vue security + patterns
laravel-inertia-vue/pattern-reviewer	Laravel+Vue conventions

Chapter 53

Appendix C: The .bee/ File Reference

A quick reference for every file and directory you will find under *.bee/*.

Path	Purpose
.bee/config.json	Project configuration
.bee/STATE.md	Current workflow state and decisions
.bee/PROJECT.md	Codebase index (structure, entry points)
.bee/CONTEXT.md	Extracted patterns and conventions
.bee/user.md	Your personal preferences (optional)
.bee/LEARNINGS.md	Cross-phase implementer learnings
.bee/false-positives.md	Dismissed review findings
.bee/COMPACT-CONTEXT.md	State snapshot for /bee:compact
.bee/AUDIT-REPORT.md	Human-readable audit findings
.bee/audit-findings.json	Machine-readable audit findings
.bee/HEALTH-REPORT.md	Output of /bee:health
.bee/specs/NNNN-slug/	A single active spec directory
.bee/specs/.../spec.md	Feature spec (behaviour only)
.bee/specs/.../phases.md	Phase breakdown
.bee/specs/.../requirements.md	Discovery Q&A record
.bee/specs/.../ROADMAP.md	Requirements-to-phase trace
.bee/specs/.../TASKS.md	Per-phase task lists with waves
.bee/specs/.../PLAN.md	Plan-phase output
.bee/specs/.../PLAN-REVIEW.md	Plan review output
.bee/specs/.../REVIEW.md	Code review findings
.bee/archive/	Completed specs, moved here on close-out
.bee/quick/	Quick task records
.bee/seeds/	Captured ideas with trigger conditions
.bee/notes/	Free-form notes
.bee/discussions/	Discuss session notes
.bee/debug/NNNN-slug/	Debug investigation session
.bee/audit-specs/	Audit-to-spec draft outputs
.bee/agents/	Project-local agents
.bee/skills/	Project-local skills

Chapter 54

Appendix D: Glossary

Agent

A specialist sub-process with one job and a specified output format, invoked by a command.

Command

A conductor that orchestrates agents and manages state for one step of the Bee lifecycle.

Conductor

Synonym for command — a workflow orchestrator that does not do work itself but directs agents that do.

Context7

A library docs provider Bee can query live for version-accurate usage examples.

Disk is truth

The Bee principle that all state lives on disk, never in conversation memory.

Finding

A structured observation by a review or audit agent, with file:line, evidence, impact, and severity.

Finding-validator

The agent that re-reads code for each raw review finding and filters out false positives.

Hook

A Claude Code lifecycle event handler. Bee uses 8 hooks to enforce invariants.

Implementation mode

economy / quality / premium — controls which model tier each agent uses.

Phase

An independently deliverable slice of a spec, with its own plan, execute, review, and commit cycle.

Research policy

required / recommended / skip — controls how aggressively the planning pipeline researches.

Scoped testing

Running only the tests relevant to a task's files so parallel agents don't block each other.

Skill

A bundled knowledge document attached conditionally to an agent's system prompt.

Spec

A markdown feature specification describing behaviour without implementation details.

SubagentStop validator

A hook-script check that runs on agent completion and enforces the output contract.

TDD iron law

Bee's rule that all implementation must follow the strict red-green-refactor cycle with test evidence.

Wave

A set of tasks that can run in parallel without stepping on each other.

Chapter 55

Appendix E: Keyboard and Endpoint Reference

Bee Hive keyboard shortcuts

Key	Action
[	Previous tab
]	Next tab
\	Toggle split pane
Escape	Close split or active tab

Bee Hive endpoints

Endpoint	Method	Purpose
/api/snapshot	GET	Full workflow state aggregator
/api/config	POST/PUT	Merge config changes
/api/file	GET	Read a .bee/ file safely

Lifecycle hook list

Hook	When
SessionStart	Session begins
PostToolUse	After Write/Edit
PreToolUse	Before Bash
PreCompact	Before /compact
SubagentStart	Agent spawns
SubagentStop	Agent finishes
Stop	Session ends
SessionEnd	Session terminates

Chapter 56

Closing Notes

You have reached the end of The Bee Book. If you have read it straight through, you should now be able to look at any Bee command and know roughly what it does, which agents it spawns, what it writes to disk, and where it fits in the lifecycle. If you used it as a reference, you should be able to find any concept by name in the index and the appendices.

Bee is a living system. The agent set and the skill list evolve from release to release. The commands themselves are stable — the 50 commands covered here are the foundation, and any future additions build on top of them rather than replacing them. The core principles in Part I are older still; they were true in Bee v1 and they will be true in Bee v5.

When in doubt, go back to the principles: spec before code, tests before implementation, disk is truth, review gates, humans commit. Every command in Bee is an expression of those five ideas. Once you internalize them, the tool gets out of your way and becomes what it is supposed to be: a quiet, disciplined collaborator that leaves a clean paper trail behind every feature it helps you build.

> *Ship with confidence, commit with care, and let the bees do the fetching.*